Buddhini Ishani OOP.docx

by WANNIGE BUDDHINI ISHANI

Submission date: 18-Sep-2026 05:06PM (UTC+0100)
Submission ID: 288653528
File name: 206445_WANNIGE_BUDDHINI_ISHANI_Buddhini_Ishani_OOP_3211962_817233210.docx (1.82M)
Word count: 5331
Character count: 31986

Assignment Cover Sheet





Cardiff Metropolitan University | Prifysgol Metropolitan Caerdydd

- Higher Diploma in Computing & Software Engineering
- CSE4205 - Object Oriented Programming

- W.Buddhini Ishani
- ST/HDCSE/CMU/57/01

Acknowledgement

I would like to extend my appreciation to all those who contributed their time, guidance and encouragement towards the successful completion of this Object-Oriented Programming assignment.

My special thanks go to my lecturer and the academic staff for their continuous guidance throughout the development of this project. The explanations, suggestions and feedback provided during the module were particularly helpful in developing a clearer understanding of Object-Oriented Programming and its practical use in software development. Their guidance also supported me in applying concepts such as classes, objects, encapsulation, inheritance, polymorphism and abstraction within the Fashion Flair Boutique management system.

I am also grateful to ICBT Campus for providing the educational facilities, learning resources and academic environment necessary to complete this assignment. The knowledge and practical understanding gained throughout the module provided a valuable foundation for designing and implementing the system.

Finally, I would like to express my appreciation to my family and friends for their continued encouragement and support during the preparation of this assignment. Their motivation and assistance helped me remain focused throughout the development and documentation process and contributed to the successful completion of this work.

Executive Summary

This assignment focuses on the analysis, design and proposed development of a Fashion Flair Boutique Management System using Object-Oriented Programming (OOP). The boutique provides clothing and accessories for men, women and children across categories such as Casual Wear, Formal Wear, Sportswear, Footwear and Accessories.

The main objective of the system is to improve product management, sales processing and user management. It supports two user roles: Sales Assistant and Boutique Manager. Sales Assistants can view, add and search products and process customer purchases, while the Boutique Manager can additionally manage staff accounts, update prices, handle promotions and view sales summaries.

The system is developed using Java and applies key OOP principles including classes, objects, encapsulation, inheritance, polymorphism and abstraction. UML diagrams and an ER diagram are used to represent the system structure, user interactions and database design.

A relational database is used to manage users, categories, products, sales and sale items. The system also uses suitable data structures to support efficient data handling. Overall, the project demonstrates how OOP, database management and appropriate data structures can be combined to develop a structured and maintainable business application.

Table of Content

Acknowledgement I

Executive Summary II

List of Tables V

List of Figures VI

Introduction VIII

TASK 1 - Object-Oriented Programming Concepts 1

1.1 Introduction to Object-Oriented Programming 1

1.2 Class 1

1.3 Object 2

1.4 Encapsulation 3

1.5 Inheritance 4

1.6 Polymorphism 6

1.7 Abstraction 7

1.8 Benefits of Object-Oriented Programming in the System 10

TASK 2 - Technical Documentation 11

2A. System Design and Diagrams 11

2B. Core System Functions 15

2C. Object-Oriented Concepts Applied in the Developed Program 33

2D. Database Operations 34

2E. User Manual 38

TASK 3 - Data Structures Used 43

3.1 Array List 43

3.2 Objects 44

3.3 Result Set 46

3.4 JTable and DefaultTableModel 47

3.5 Comparison of Data Structures 50

3.6 Efficiency, Scalability and Suitability 50

Conclusion 52

References 53

List of Tables

Table 1 : Database Roles..... 38

List of Figures

Figure 1 : Product class and its attributes 2
Figure 2 : Product Object Creation 3
Figure 3 : Encapsulation in Product Class 4
Figure 4 : Inheritance between User and User Roles 6
Figure 5 : Runtime Polymorphism in OOPTest 7
Figure 6 : Different login behaviours produced by polymorphic objects 7
Figure 7 : Abstraction using Manageable Interface 9
Figure 8 : Use Case Diagram of Fashion Flair Boutique 12
Figure 9 : Class Diagram of Fashion Flair Boutique 13
Figure 10 : Purchase Processing Sequence Diagram 14
Figure 11 : Entity Relationship Diagram of Database 15
Figure 12 : Fashion Flair Boutique login interface 16
Figure 13 : Login Successful Message 16
Figure 14 : Sales Assistant Dashboard 17
Figure 15 : Product Management and Product List 18
Figure 16 : Product details 20
Figure 17 : Add Product Interface 21
Figure 18 : Product Added Successfully 22
Figure 19 : Product record in Database 23
Figure 20 : Selecting a Product for Update 24
Figure 21 : Search Products by Category 26
Figure 22 : Process Customer Purchase Interface 28
Figure 23 : Sales Table Records 28
Figure 24 : SalesItem Tables Records 28
Figure 25 : Product Stock Updated After a Purchase 29
Figure 26 : Manage Sales Assistant Accounts 30
Figure 27 : Product Price Update Interface 31
Figure 28 : Promotional Discount Interface 32
Figure 29 : Daily and Monthly Sales Summary 33

Figure 30 : Database Connection Class 36
Figure 31 : Database Connection Test 36
Figure 32 : Fashion Flair Boutique Database Tables 38
Figure 33 : Login Form..... 39
Figure 34 : Sales Assistant Dahboard 40
Figure 35 : Boutique Manager Dashboard..... 42
Figure 36 : ArrayList Used for Product Records 44
Figure 37 : Product Object Structure 45
Figure 38 : ResultSet Used for Database Retrieval 47
Figure 39 : Product Records Displayed Using JTable 49

Introduction

Fashion Flair Boutique is a clothing and accessories store that provides fashion products for men, women and children. The boutique offers different categories such as Casual Wear, Formal Wear, Sportswear, Footwear and Accessories. As the number of products and customer purchases increases, manually managing product information, stock and sales becomes difficult. Therefore, an automated boutique management system was developed to support the daily activities of the boutique.

The developed system is a Java-based desktop application created using Object-Oriented Programming principles. It provides different functions according to the responsibilities of the Sales Assistant and Boutique Manager. Sales Assistants can view products, add new products, search products by category and process customer purchases. The Boutique Manager has access to these functions and can additionally manage Sales Assistant accounts, update product prices, apply promotional discounts and view sales summaries.

The system also uses a Microsoft Access database to store and retrieve application data. UCanAccess is used to establish communication between the Java application and the Access database. Database Access is separated into DAO classes so that database operations can be managed independently from the user interface.

The main purpose of this project is not only to provide the required boutique functions but also to demonstrate practical use of Object-Oriented Programming concepts. Classes, objects, encapsulation, inheritance, polymorphism and abstraction are incorporated into the application. Data structures such as ArrayList are also used to handle collections of products and users efficiently.

This documentation explains the development of the Fashion Flair Boutique system according to Tasks 1, 2 and 3. It covers the theoretical OOP concepts, system diagrams, implemented functions, database operations, user interface and data structures used in the application.

Project Source Code:

Project Repository: Fashion Flair Boutique – GitHub Repository

https://github.com/buddhiniishani6613-lab/FashionFlairBoutique

TASK 1 - Object-Oriented Programming Concepts

1.1 Introduction to Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming approach where a software system is designed using objects and classes. An object represents an individual entity with its own data and behaviour, while a class defines the structure and behaviour that objects created from it can have.

OOP is suitable for the Fashion Flair Boutique system because the application contains several real-world entities such as users, products, sales and categories. These entities can be represented using Java classes. For example, a product has information such as product name, brand, size, colour, price and stock quantity. A Product class can therefore represent a product in the application.

Using OOP also makes the system easier to organize because different responsibilities can be separated into different classes. This reduces unnecessary duplication and makes the application easier to maintain.

1.2 Class

A class is a blueprint used to create objects. It defines the attributes and methods that objects of that class can contain.

In the developed system, Product, User, SalesAssistant and BoutiqueManager are examples of classes.

The Product class represents fashion products stored by the boutique. It contains attributes such as productID, productName, brand, categoryID, size, color, price and stockQuantity.

- Real-world Example:

  A product record in a boutique can be considered an object, while the general product design can be considered the class.

Product.java

Source History

```java
package model;

public class Product implements Manageable {
    @Override
public void add() {
    System.out.println("Product added successfully.");
}

@Override
public void update() {
    System.out.println("Product updated successfully.");
}

@Override
public void delete() {
    System.out.println("Product deleted successfully.");
}

    private int productID;
    private String productName;
    private String brand;
    private int categoryID;
    private String size;
    private String color;
    private double price;
    private int stockQuantity;
```

Figure 1 : Product class and its attributes

1.3 Object

An object is an instance of a class. When a class is created as an object, the object receives its own values for the attributes defined by the class.

For example, the database contains a product called Cotton T-Shirt from Nike. In the Java program, this information can be represented using a Product object. Similarly, a logged-in user is represented using a User object.

The application also demonstrates object creation when opening forms. For example, when the manager selects Manage Products, a ManageProductsForm object is created and displayed.

- Real-world Example:
  A specific Nike Cotton T-Shirt is an object, while Product is the class describing all products.

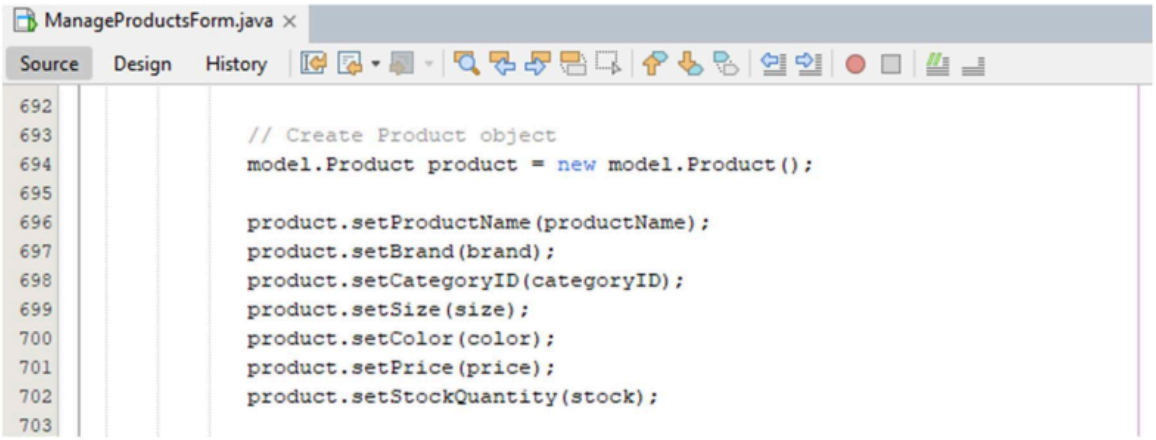


Figure 2 : Product Object Creation

1.4 Encapsulation

Encapsulation means keeping the internal data of a class protected and providing controlled access through methods. In Java, this is commonly achieved by declaring attributes as private and using public getter and setter methods.

The User and Product classes in the developed system apply encapsulation. For example, product information such as price and stock quantity is declared as private. Other classes cannot directly change these variables. Instead, getter and setter methods are used.

The Product class also performs simple validation inside its setter methods. The price and stock quantity are only changed when the supplied value is not negative. This provides a basic level of data protection.

- Real-world Example:
  A boutique customer cannot directly modify the internal stock record. Only an authorized system function should update it.

Product.java

Source History

```
93      public double getPrice() {
94          return price;
95      }
96
97      public void setPrice(double price) {
98          if (price >= 0) {
99              this.price = price;
100         }
101     }
102
103     public int getStockQuantity() {
104         return stockQuantity;
105     }
106
107     public void setStockQuantity(int stockQuantity) {
108         if (stockQuantity >= 0) {
109             this.stockQuantity = stockQuantity;
110         }
111     }
```

Figure 3 : Encapsulation in Product Class

1.5 Inheritance

Inheritance allows one class to acquire properties and behaviours from another class. It supports code reuse and allows related classes to be organized into a hierarchy.

In the project, SalesAssistant and BoutiqueManager inherit from the User class. Therefore, both roles can use common user information such as user ID, full name, username, password and role.

The statement SalesAssistant extends User represents this relationship. Similarly, BoutiqueManager extends User.

This is appropriate because both types of users are system users, but they have different responsibilities.

- Real-world Example:
  A Sales Assistant and Boutique Manager are both employees/users of the boutique, but their responsibilities are different.

SalesAssistant.java

Source History

```
1  package model;
2
3  public class SalesAssistant extends User {
4
5      public SalesAssistant() {
6          super();
7      }
8
9      public void addProduct() {
10         System.out.println("Sales Assistant can add new products.");
11     }
12
13     public void searchProduct() {
14         System.out.println("Sales Assistant can search products.");
15     }
16
17     public void processPurchase() {
18         System.out.println("Sales Assistant can process customer purchases.");
19     }
20
21     @Override
23     public void login() {
24         System.out.println("Sales Assistant logged in.");
25     }
}
```

BoutiqueManager.java

Source History

```java
package model;

public class BoutiqueManager extends User {

    public BoutiqueManager() {
        super();
    }

    public void manageUsers() {
        System.out.println("Manager can create and manage Sales Assistant accounts.");
    }

    public void updatePrices() {
        System.out.println("Manager can update product prices.");
    }

    public void manageDiscounts() {
        System.out.println("Manager can manage seasonal discounts and promotions.");
    }

    public void viewSalesSummary() {
        System.out.println("Manager can view daily and monthly sales summaries.");
    }

    @Override
    public void login() {
        System.out.println("Boutique Manager logged in.");
    }
}
```

Figure 4 : Inheritance between User and User Roles

1.6 Polymorphism

Polymorphism means that one common interface or parent reference can represent different objects and allow different implementations of the same method.

The project demonstrates runtime polymorphism through the login() method. The User class contains a login() method. Both SalesAssistant and BoutiqueManager override this method with their own implementation.

In OOPTest, a User reference is used to hold a SalesAssistant object and a BoutiqueManager object. When login() is called, Java executes the overridden method according to the actual object.

This allows common user handling while still supporting role-specific behaviour.

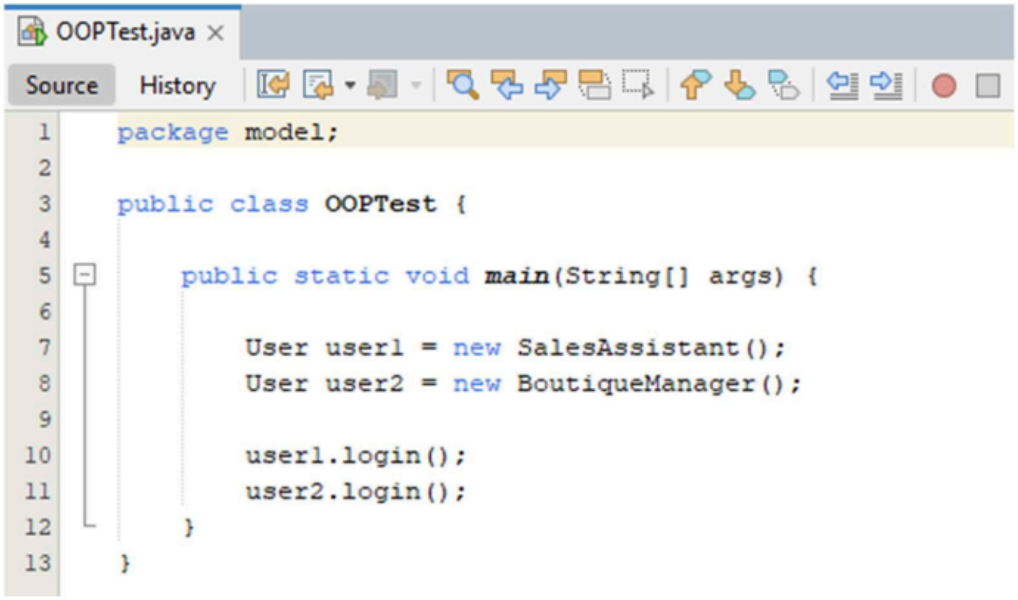


Figure 5 : Runtime Polymorphism in OOPTest

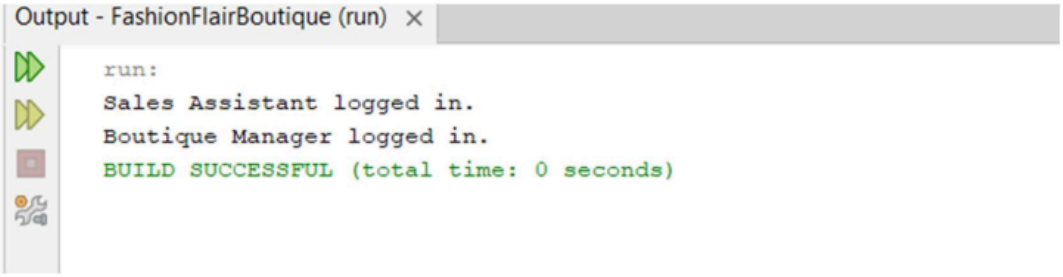


Figure 6 : Different login behaviours produced by polymorphic objects

1.7 Abstraction

Abstraction means representing the important operations of an entity while hiding unnecessary implementation details. It allows a program to focus on what an object can do rather than how every operation is internally performed.

The project applies abstraction through the Manageable interface. The interface defines common operations such as add(), update() and delete().

The Product class implements the Manageable interface and provides implementations for these methods. The interface therefore defines the required behaviour while the implementing class provides the actual behaviour.

- **Real-world Example:**

  A boutique management system may define that a product can be added, updated or deleted without requiring the user to understand the database commands behind those operations.

Manageable.java

```java
package model;

public interface Manageable {

    void add();

    void update();

    void delete();
}
```

Product.java

```java
package model;

public class Product implements Manageable {
    @Override
public void add() {
    System.out.println("Product added successfully.");
}

@Override
public void update() {
    System.out.println("Product updated successfully.");
}

@Override
public void delete() {
    System.out.println("Product deleted successfully.");
}

    private int productID;
    private String productName;
    private String brand;
    private int categoryID;
    private String size;
    private String color;
    private double price;
    private int stockQuantity;

    public Product() {
    }

    public Product(int productID, String productName, String brand,
                   int categoryID, String size, String color,
                   double price, int stockQuantity) {

        this.productID = productID;
        this.productName = productName;
        this.brand = brand;
        this.categoryID = categoryID;
        this.size = size;
        this.color = color;
        this.price = price;
        this.stockQuantity = stockQuantity;
```

Figure 7 : Abstraction using Manageable Interface

1.8 Benefits of Object-Oriented Programming in the System

The use of OOP provides several benefits to the Fashion Flair Boutique system.

- Reusability

Inheritance allows common user functionality to be defined once in the User class and reused by the SalesAssistant and BoutiqueManager classes.

- Data Protection

Encapsulation protects important attributes by keeping them private and providing controlled access through methods.

- Flexibility

Polymorphism allows different user types to provide their own implementation of common operations such as login().

- Reduced Complexity

Abstraction allows the system to define required operations through an interface without exposing unnecessary implementation details.

- Maintainability

Separating the system into classes makes individual components easier to understand, modify and maintain.

- Extensibility

The object-oriented structure allows additional user types, product-related classes or manageable entities to be introduced without redesigning the entire application.

TASK 2 - Technical Documentation

2A. System Design and Diagrams

The Fashion Flair Boutique system was designed using the requirements given in the scenario. The system has two main user levels: Sales Assistant and Boutique Manager.

The Sales Assistant can view products, add products, search products by category and process purchases. The Boutique Manager can perform these functions and additionally manage staff accounts, update prices, manage promotions and view sales summaries.

2A.1 Use Case Diagram

The Use Case Diagram represents the interaction between system users and the main functions of the application.

The two actors are Sales Assistant and Boutique Manager. The Sales Assistant interacts with View Products, Add Product, Search by Category and Process Purchase. The Boutique Manager has access to the Sales Assistant functions and additionally interacts with Manage Staff, Update Prices, Manage Promotions and View Sales Summary.

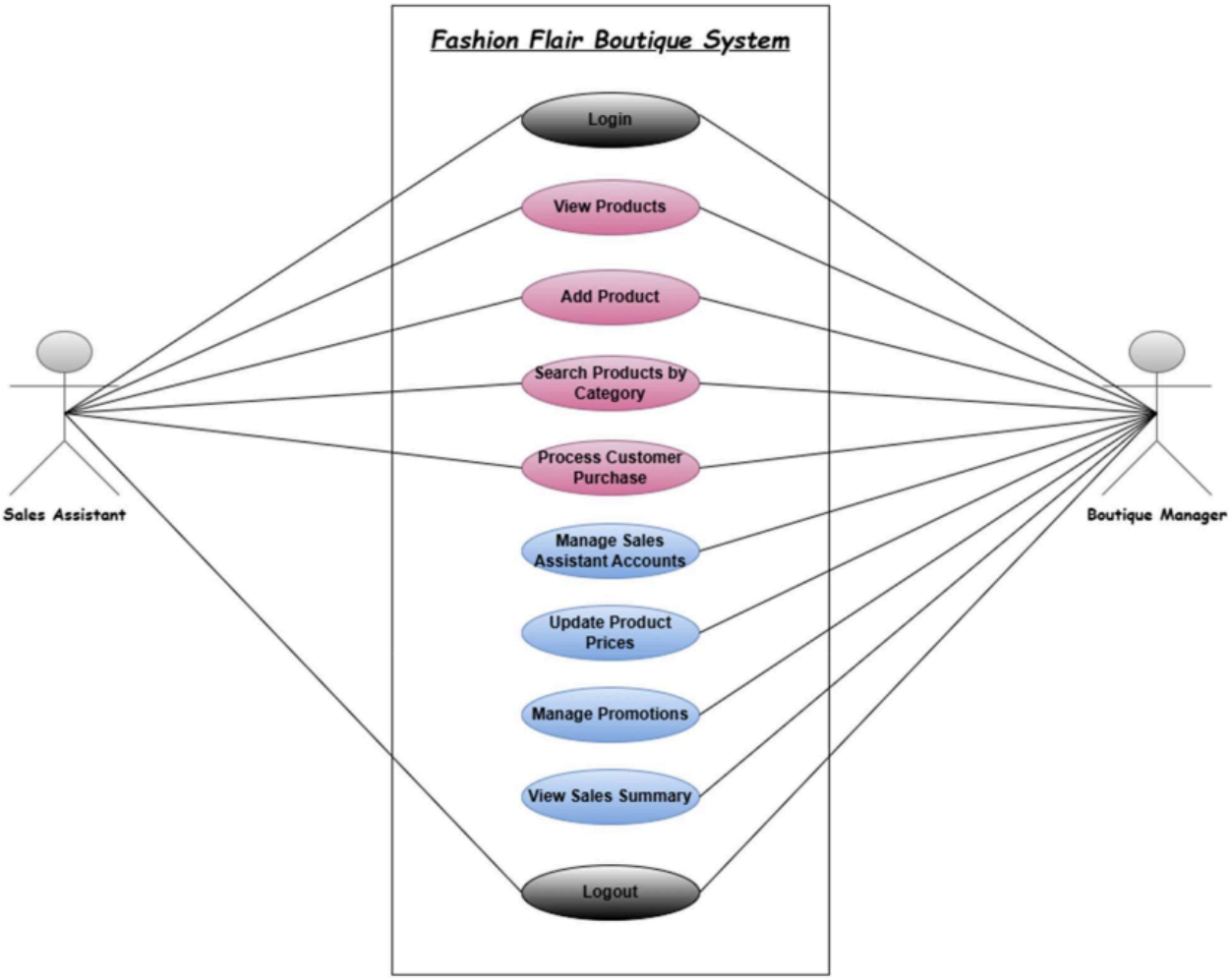


Figure 8 : Use Case Diagram of Fashion Flair Boutique

2A.2 Class Diagram

The Class Diagram represents the main classes and relationships in the application.

The User class is the parent class for SalesAssistant and BoutiqueManager. The Product class represents products and implements the Manageable interface. DAO classes such as ProductDAO, CategoryDAO, UserDAO and SaleDAO are responsible for database operations.

The class diagram therefore demonstrates both the object-oriented design and separation of responsibilities used in the application.

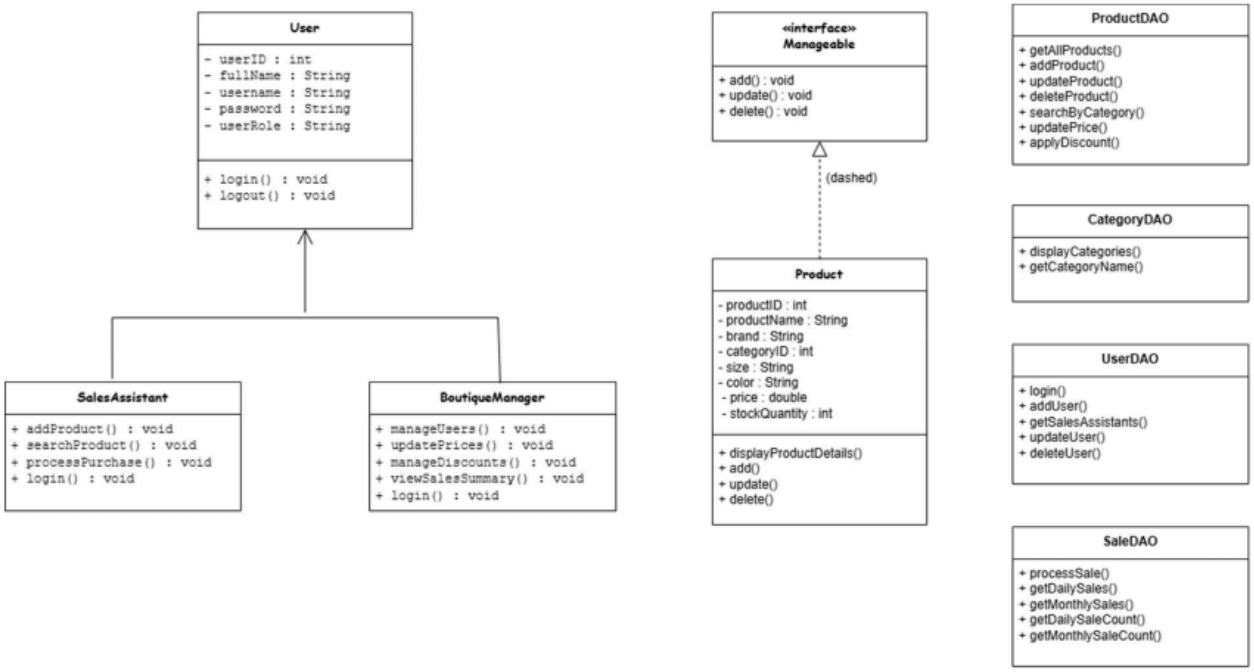


Figure 9 : Class Diagram of Fashion Flair Boutique

2A.3 Sequence Diagram

The Sequence Diagram describes how objects communicate during an operation. A suitable example for this project is the customer purchase process.

The user selects the Process Purchase option. The application receives product and quantity information, communicates with SaleDAO, creates the sale record and sale item record, updates the product stock and completes the transaction.

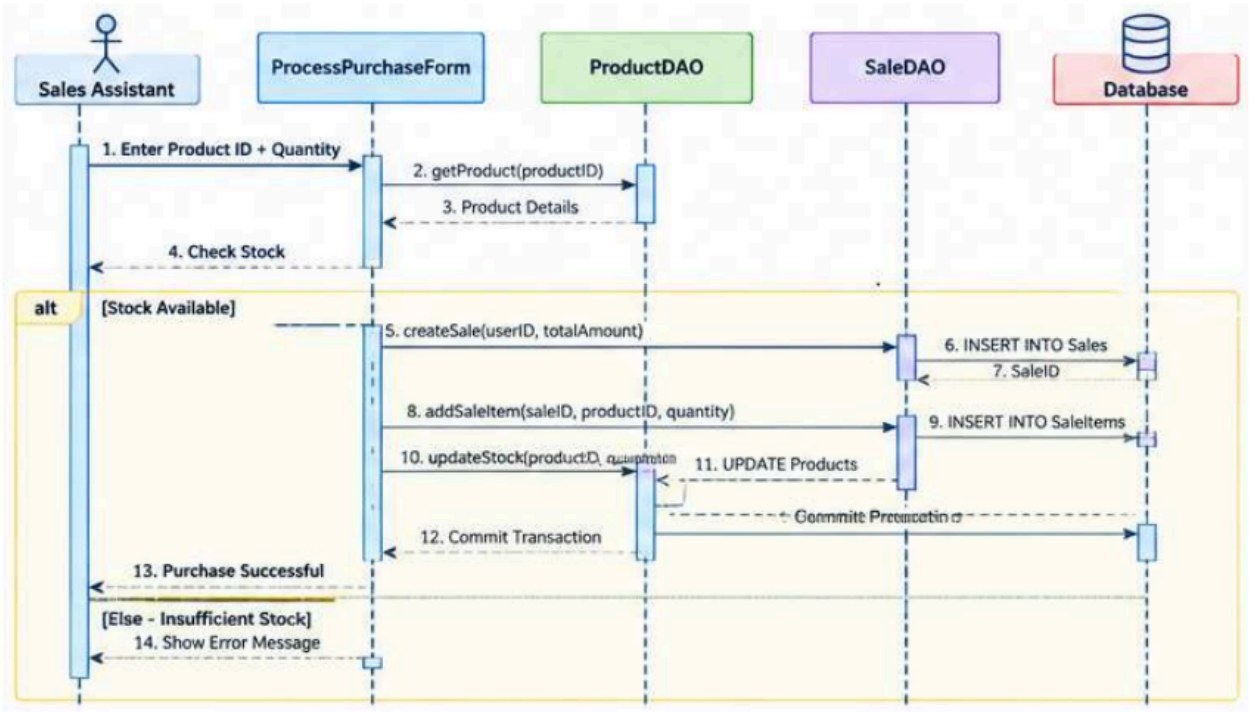


Figure 10 : Purchase Processing Sequence Diagram

2A.4 Entity Relationship Diagram

The database contains the main tables required for the application. The major tables are Users, Categories, Products, Sales and SaleItems.

A user can be associated with multiple sales. A category can contain multiple products. A sale can contain sale items, while each sale item refers to a product.

The database design separates product information, user information and sales information instead of storing all information in one table.



Figure 11 : Entity Relationship Diagram of Database

2B. Core System Functions

The Fashion Flair Boutique application implements the core functions required for the two user levels. The system separates general user operations from management-specific operations through the use of role-based classes and interfaces.

The main implemented functions include user login, viewing products, adding products, searching products by category, processing purchases, managing Sales Assistant accounts, updating product prices, applying promotional discounts and viewing sales summaries.

2B.1 User Login

The login function allows registered users to access the system using a username and password. The LoginForm receives the login details and sends them to UserDAO.

UserDAO uses a parameterized SQL query to search the Users table. If matching information is found, a User object is created and returned to the login form.

The application then checks the user's role. A Boutique Manager is directed to ManagerDashboard, while a Sales Assistant is directed to SalesAssistantDashboard.

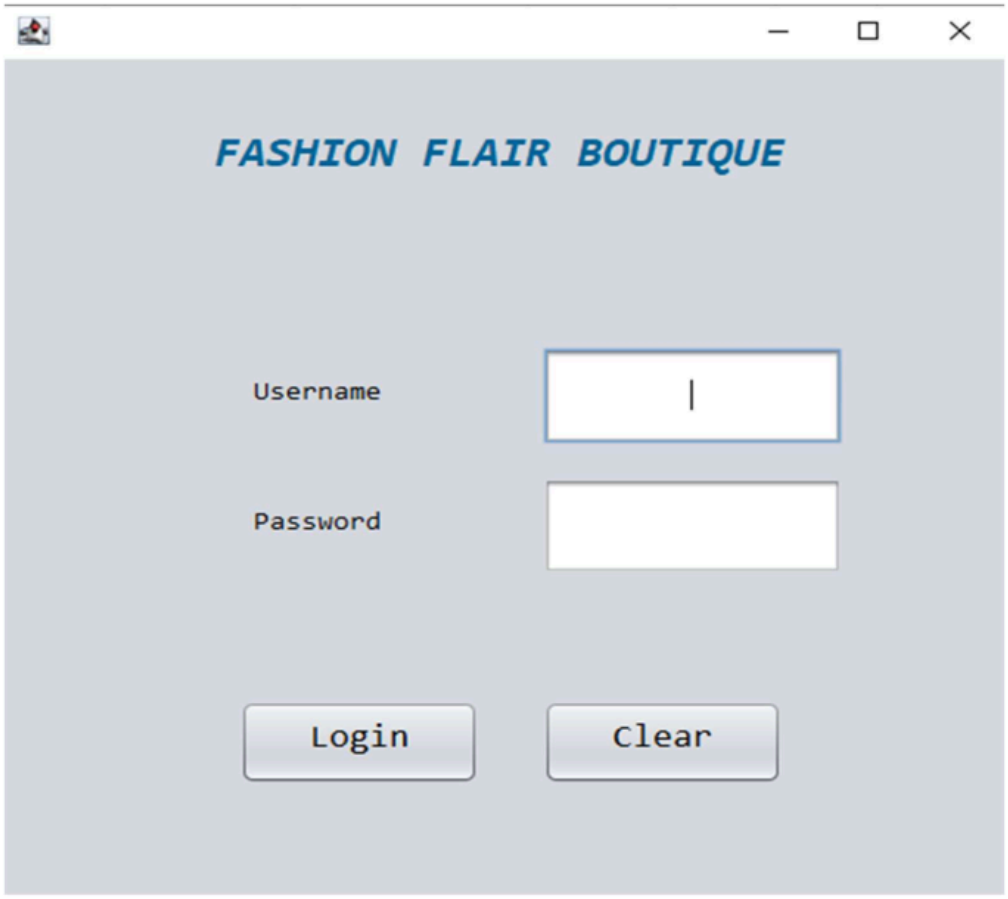


Figure 12 : Fashion Flair Boutique login interface

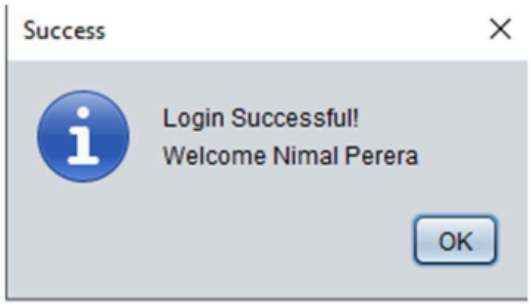


Figure 13 : Login Successful Message

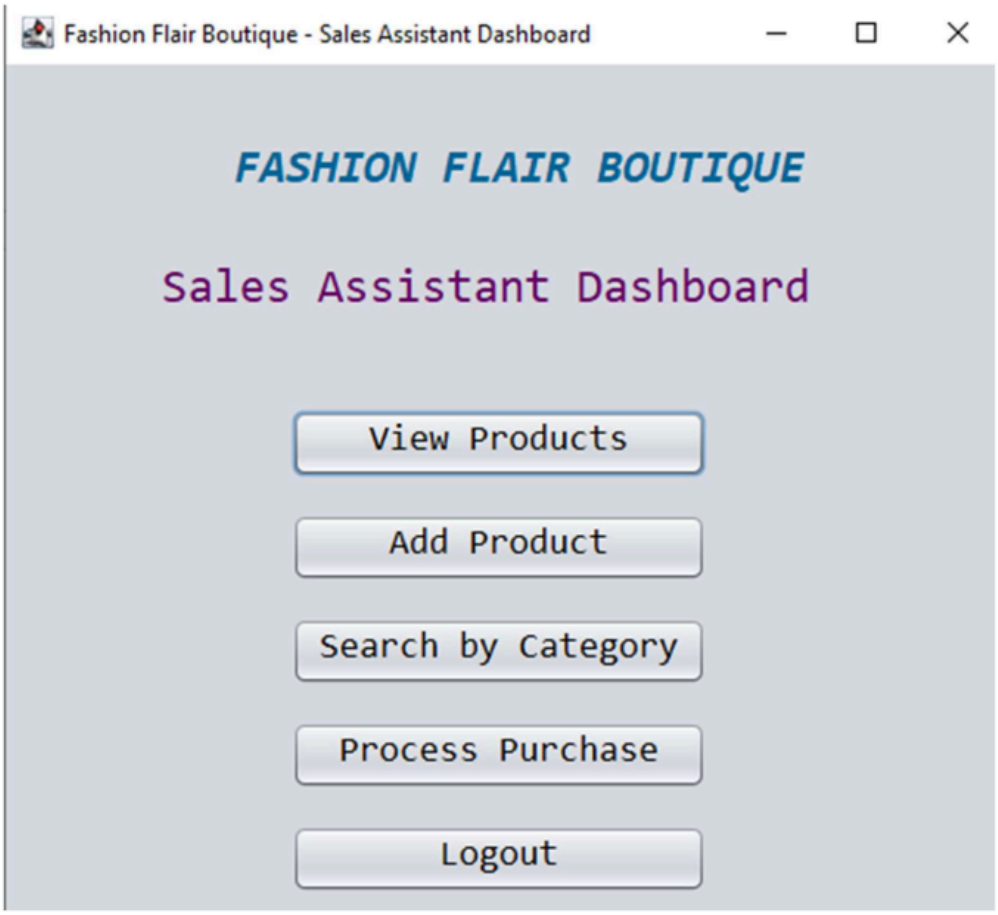


Figure 14 : Sales Assistant Dashboard

2B.2 View Products

The View Products function retrieves all available products from the database.

ProductDAO.getAllProducts() executes a SELECT query and creates Product objects from the returned records. The objects are stored in an ArrayList and displayed through the application.

This separates database retrieval from the user interface.

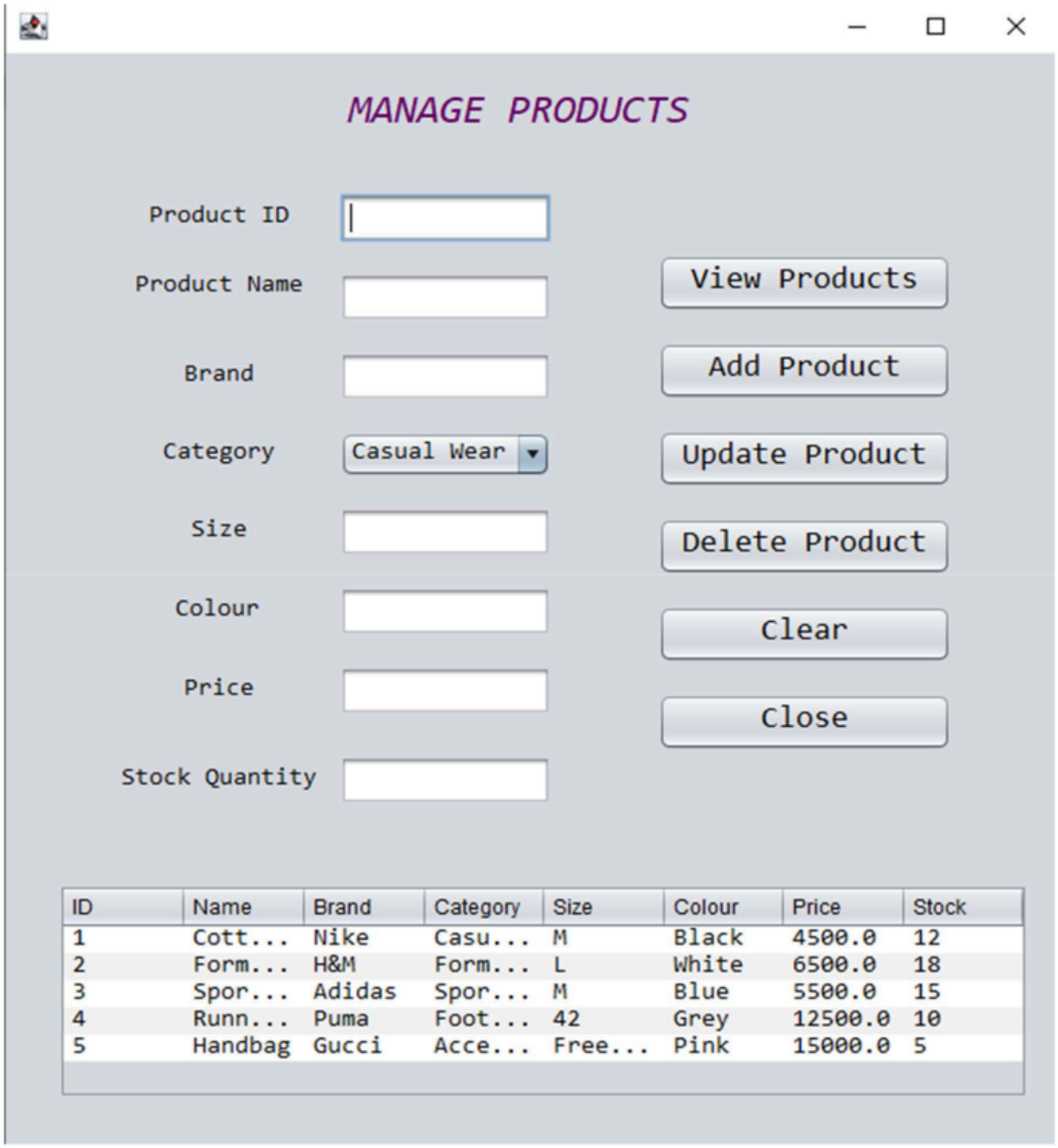
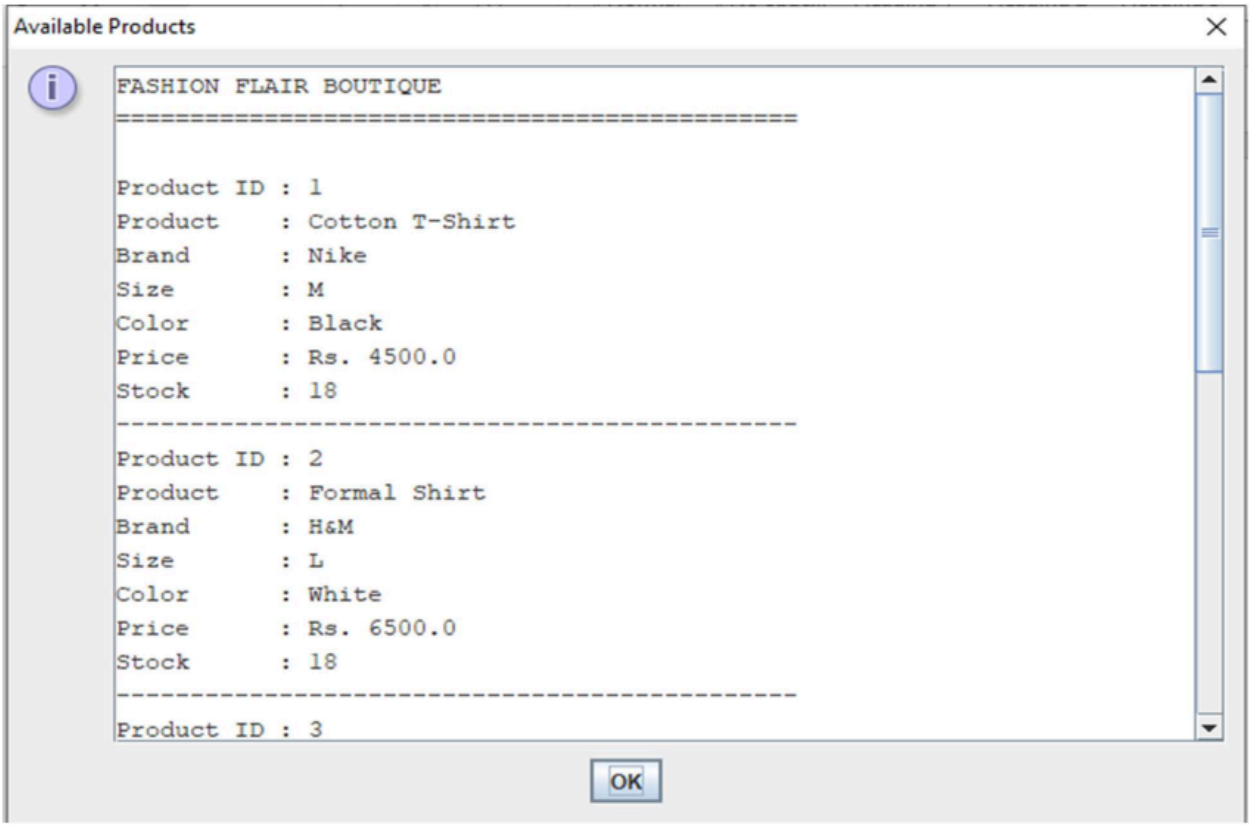
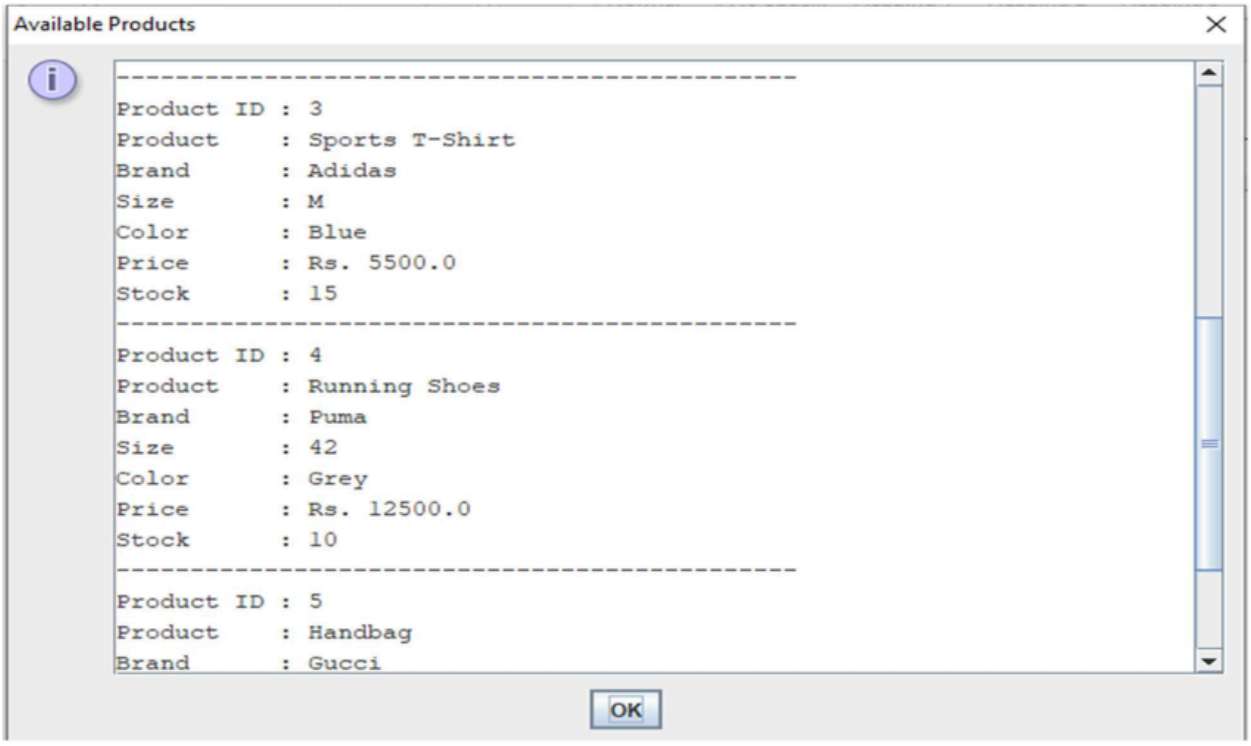


Figure 15 : Product Management and Product List


Available Products
FASHION FLAIR BOUTIQUE
==========
Product ID : 1
Product : Cotton T-Shirt
Brand : Nike
Size : M
Color : Black
Price : Rs. 4500.0
Stock : 18

Product ID : 2
Product : Formal Shirt
Brand : H&M
Size : L
Color : White
Price : Rs. 6500.0
Stock : 18

Product ID : 3
OK


Available Products

Product ID : 3
Product : Sports T-Shirt
Brand : Adidas
Size : M
Color : Blue
Price : Rs. 5500.0
Stock : 15

Product ID : 4
Product : Running Shoes
Brand : Puma
Size : 42
Color : Grey
Price : Rs. 12500.0
Stock : 10

Product ID : 5
Product : Handbag
Brand : Gucci
OK

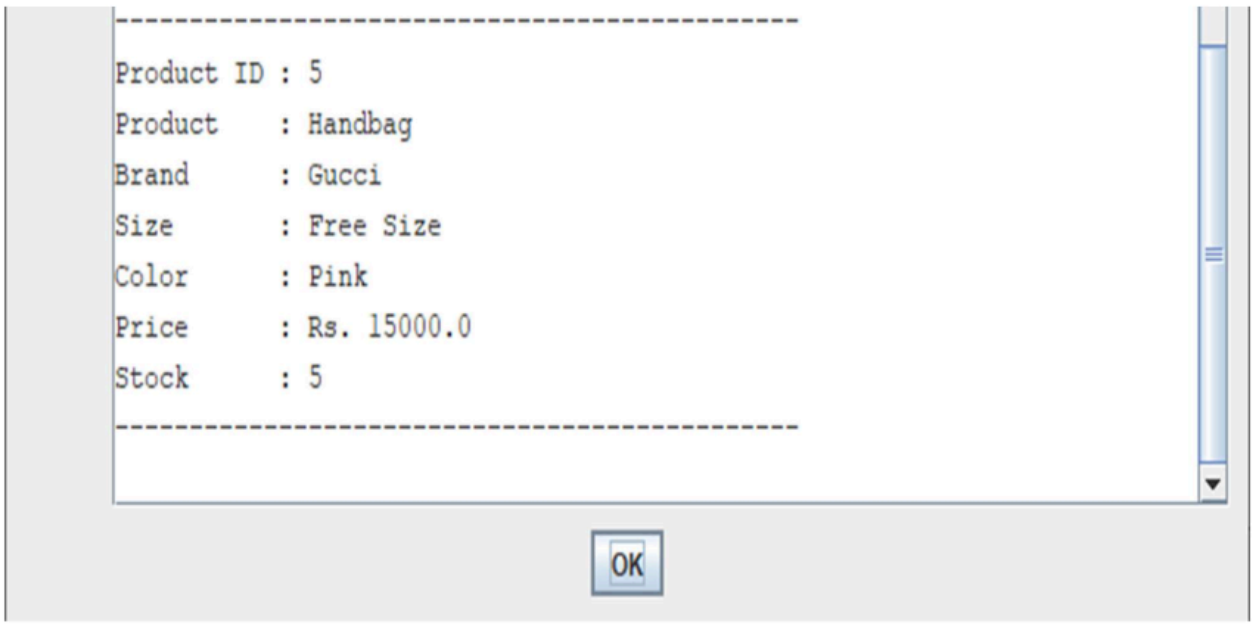


Figure 16 : Product details

2B.3 Add Product

The Add Product function allows staff to enter product information including product name, brand, category, size, colour, price and stock.

The form validates required fields and numeric values before creating a Product object. The object is then passed to ProductDAO.addProduct().

The database receives the new product using an INSERT operation.

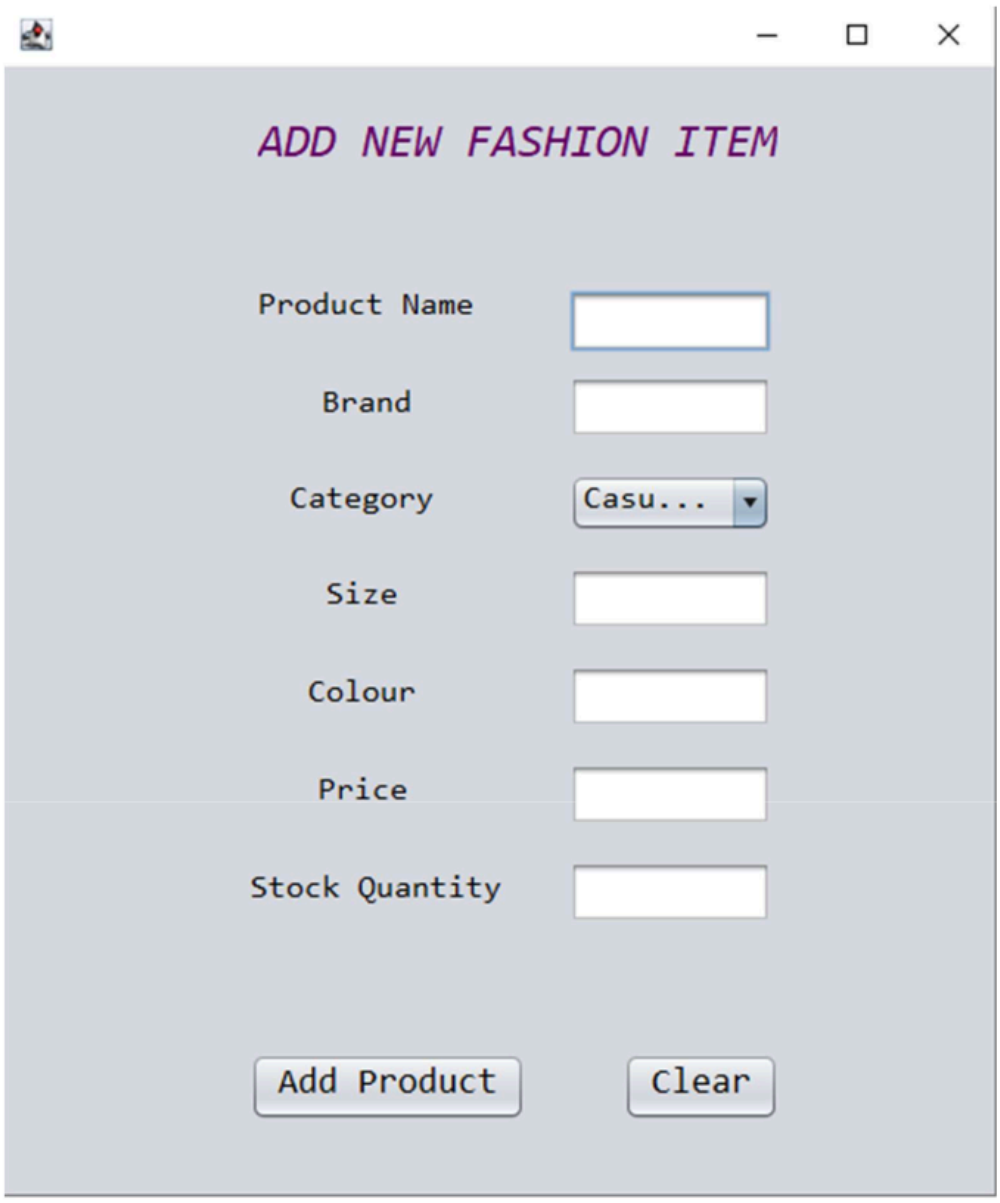


Figure 17 : Add Product Interface

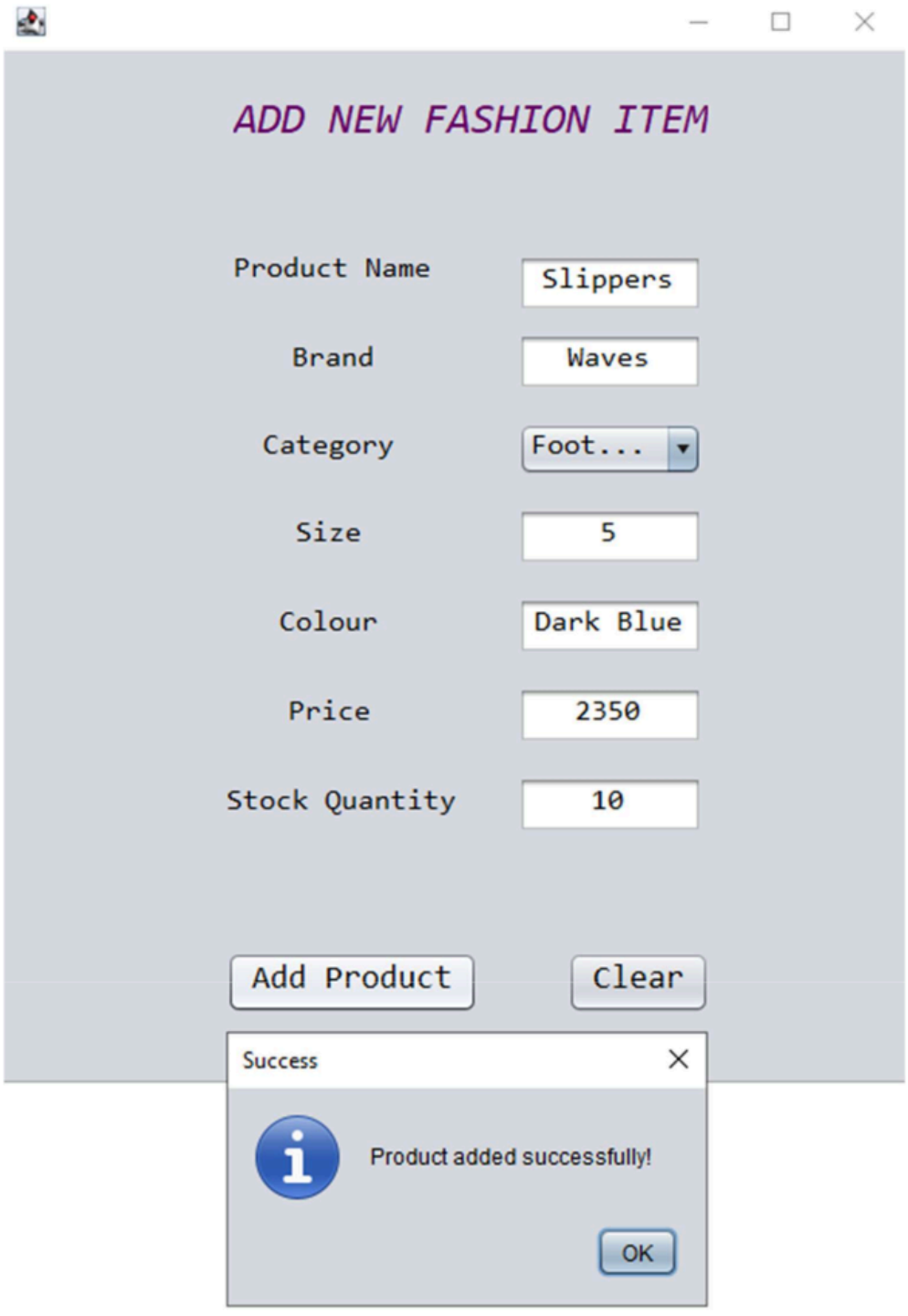


Figure 18 : Product Added Successfully

Products

ProductID	ProductNam	Brand	CategoryID	Size	Color	Price	StockQuanti	Click to Add
1	Cotton T-Shirt	Nike	1	M	Black	$4,500.00	12	
2	Formal Shirt	H&M	2	L	White	$6,500.00	18	
3	Sports T-Shirt	Adidas	3	M	Blue	$5,500.00	15	
4	Running Shoes	Puma	4	42	Grey	$12,500.00	10	
5	Handbag	Gucci	5	Free Size	Pink	$15,000.00	5	
6	Slippers	Waves	4	5	Dark Blue	$2,350.00	10	
(New)			0			$0.00	0	

Figure 19 : Product record in Database

2B.4 Update and Delete Product

The Manager's Manage Products interface provides product update and delete operations.

When a row is selected, the product information is loaded into the input fields. The manager can modify the details and click Update Product. The system sends the updated Product object to ProductDAO.updateProduct().

For deletion, the selected Product ID is used by ProductDAO.deleteProduct(). A confirmation message is displayed before deleting the selected product.

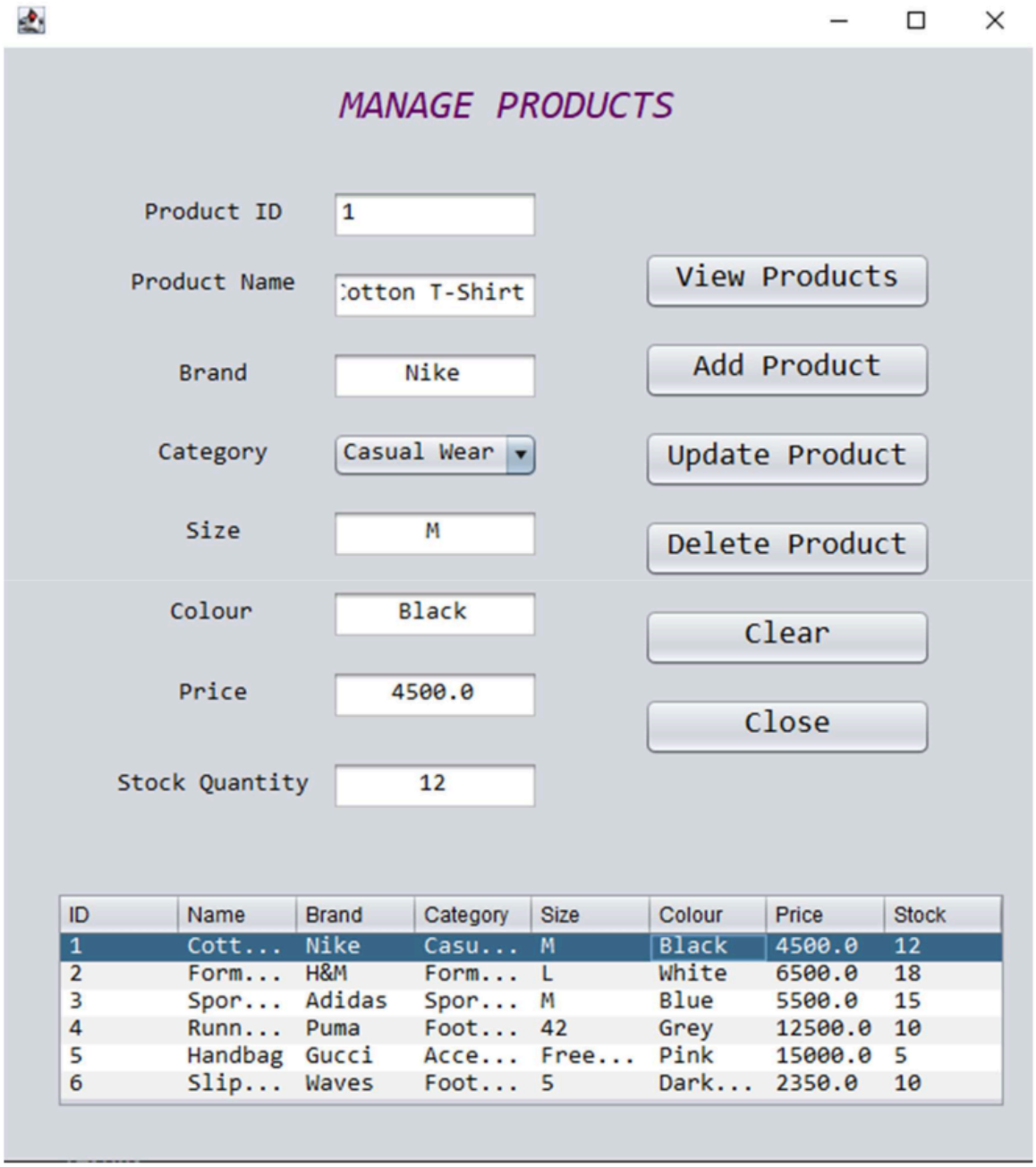


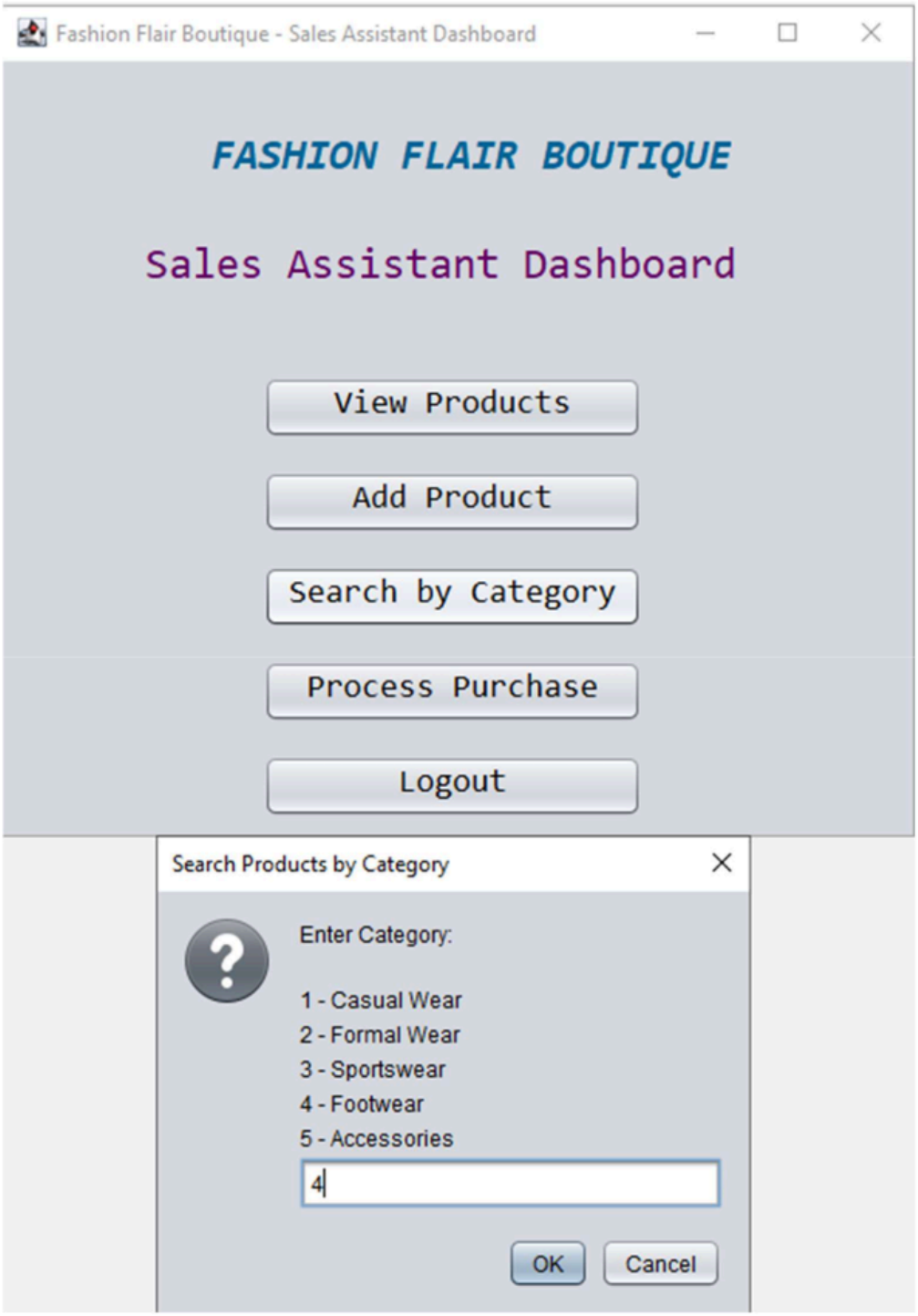
Figure 20 : Selecting a Product for Update

2B.5 Search Products by Category

The category search function allows users to find products according to categories such as Casual Wear, Formal Wear, Sportswear, Footwear and Accessories.

The selected category ID is passed to ProductDAO.searchByCategory(). The method retrieves matching products and returns them as an ArrayList<Product>.

This makes it easier to find products without viewing the complete product list.



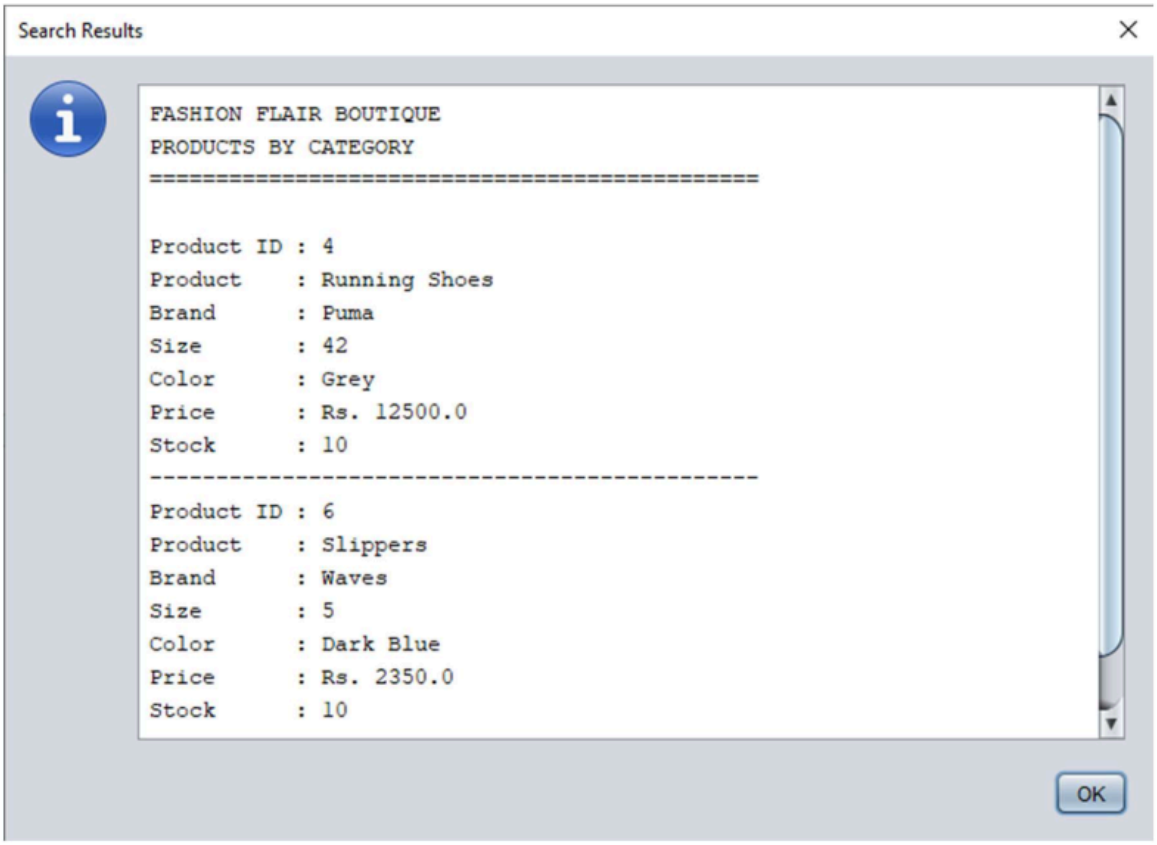


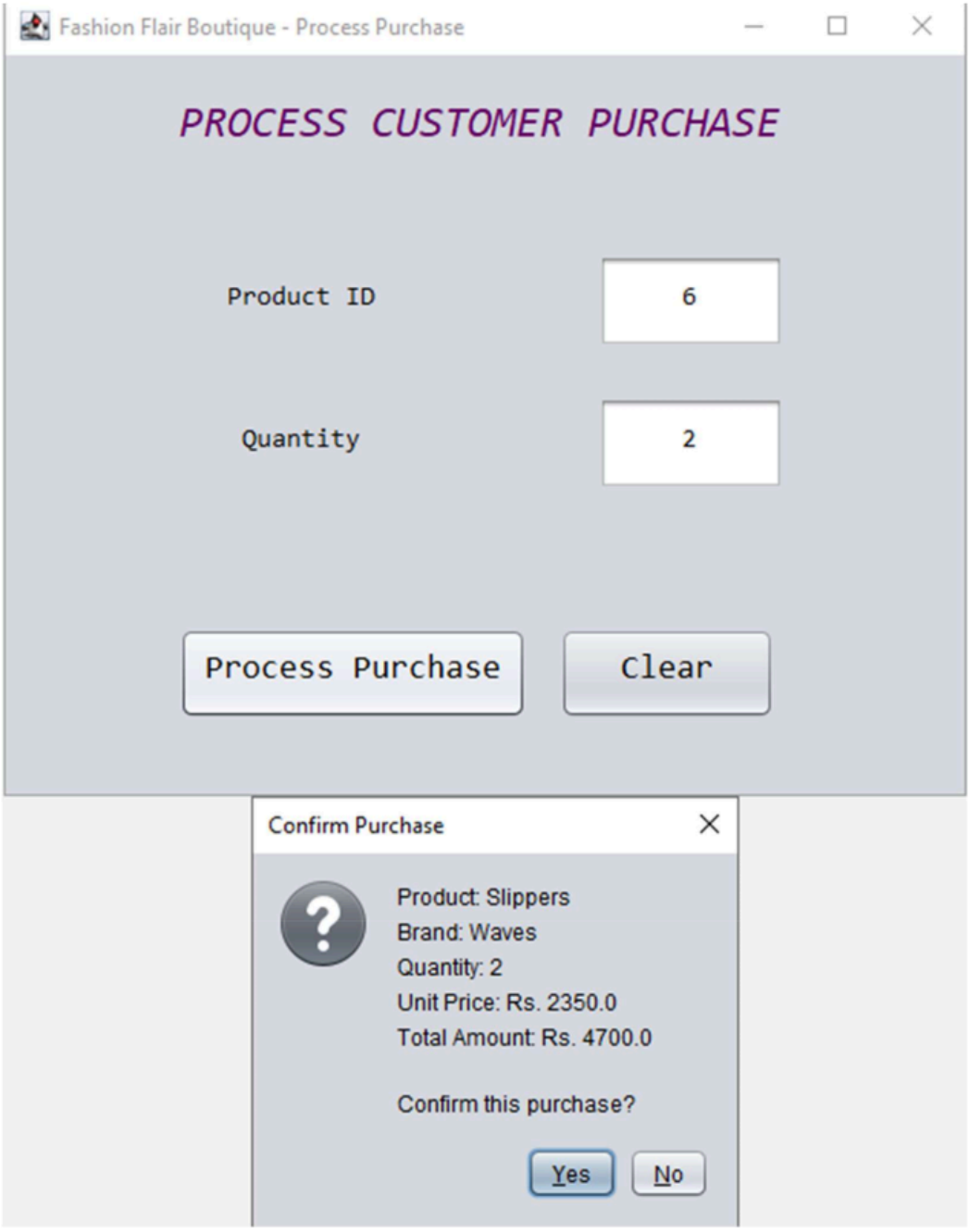
Figure 21 : Search Products by Category

2B.6 Process Customer Purchase

The Process Purchase function records customer purchases in the database.

The system records the sale date, user, total amount, product, quantity and unit price. It also decreases the product stock after a successful purchase.

The SaleDAO uses a database transaction so that the related sale, sale item and stock update are processed together. If one operation fails, the transaction can be rolled back.


Fashion Flair Boutique - Process Purchase
PROCESS CUSTOMER PURCHASE
Product ID
6
Quantity
2
Process Purchase
Clear
Confirm Purchase
Product: Slippers
Brand: Waves
Quantity: 2
Unit Price: Rs. 2350.0
Total Amount: Rs. 4700.0
Confirm this purchase?
Yes
No

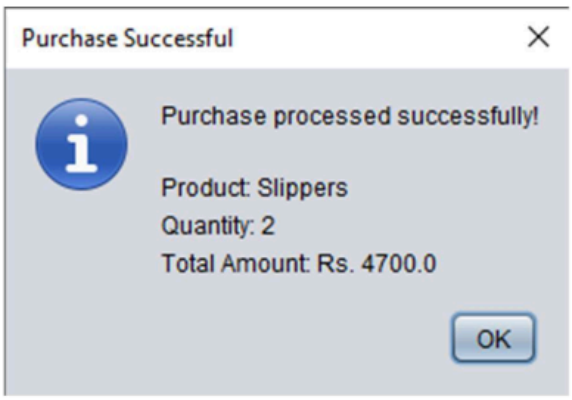


Figure 22 : Process Customer Purchase Interface

Sales

SaleID	SaleDate	UserID	TotalAmoun	Click to Add
1	##########	1	$9,000.00	
2	##########	1	$13,500.00	
3	##########	1	$13,500.00	
4	##########	1	$4,700.00	
(New)		0	$0.00	

Figure 23 : Sales Table Records

SaleItems

SaleItemID	SaleID	ProductID	Quantity	UnitPrice	Click to Add
1	1	1	2	$4,500.00	
2	2	1	3	$4,500.00	
3	3	1	3	$4,500.00	
4	4	6	2	$2,350.00	
(New)	0	0	0	$0.00	

Figure 24 : SalesItem Tables Records

Products

ProductID	ProductNam	Brand	CategoryID	Size	Color	Price	StockQuanti	Click to Add
1	Cotton T-Shirt	Nike	1	M	Black	$4,500.00	12	
2	Formal Shirt	H&M	2	L	White	$6,500.00	18	
3	Sports T-Shirt	Adidas	3	M	Blue	$5,500.00	15	
4	Running Shoes	Puma	4	42	Grey	$12,500.00	10	
5	Handbag	Gucci	5	Free Size	Pink	$15,000.00	5	
6	Slippers	Waves	4	5	Dark Blue	$2,350.00	8	
(New)			0			$0.00	0	

Figure 25 : Product Stock Updated After a Purchase

2B.7 Sales Assistant Account Management

The Boutique Manager can create and manage Sales Assistant accounts through StaffManagementForm.

The manager enters the staff member's full name, username and password. UserDAO.addUser() inserts the new user into the Users table.

Existing Sales Assistant accounts are retrieved through getSalesAssistants() and displayed in the table.

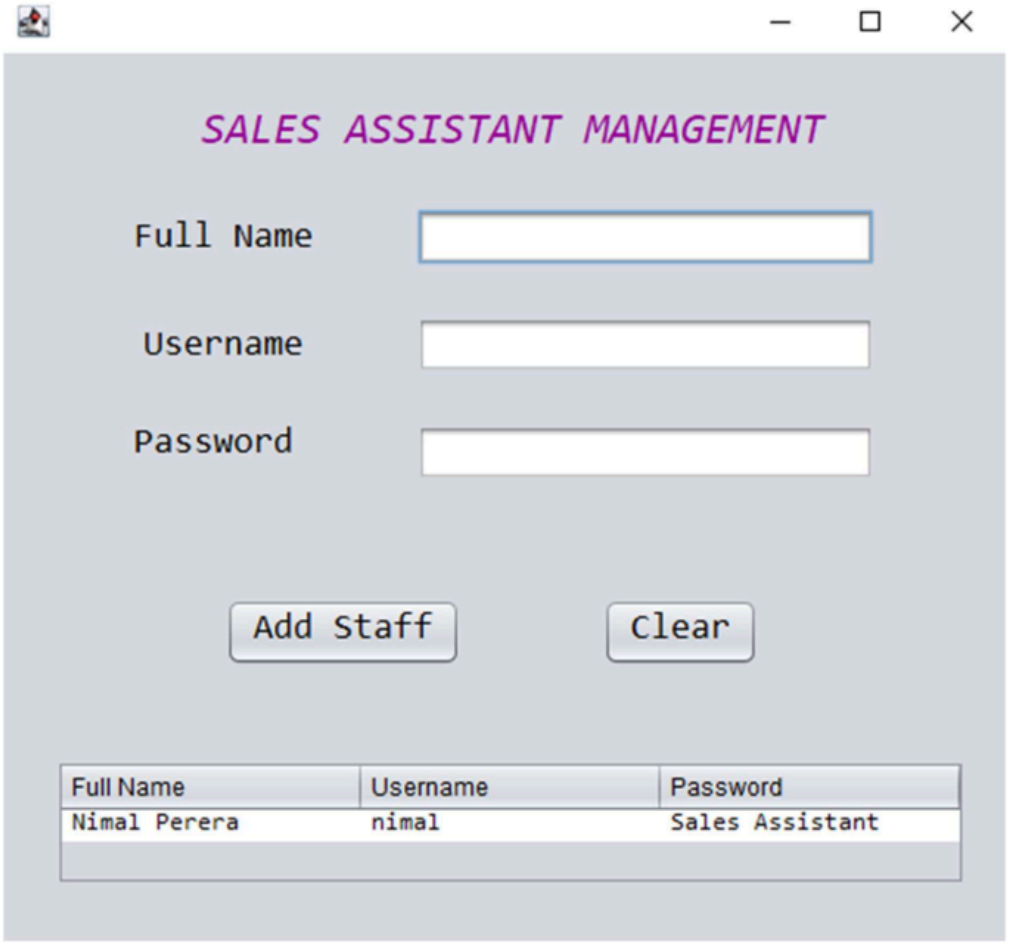


Figure 26 : Manage Sales Assistant Accounts

2B.7 Update Product Prices

The UpdatePriceForm allows the Boutique Manager to update the price of an existing product.

The manager enters the Product ID and new price. The application validates the values before calling ProductDAO.updatePrice().

This function provides a direct way to maintain current product prices.



Figure 27 : Product Price Update Interface

2B.8 Promotional Discounts

The Promotion Form allows the Boutique Manager to apply a percentage discount to a selected product.

The manager provides the Product ID and discount percentage. The application calculates the reduced price and updates the product price in the database.

This function supports promotional pricing within the boutique system.

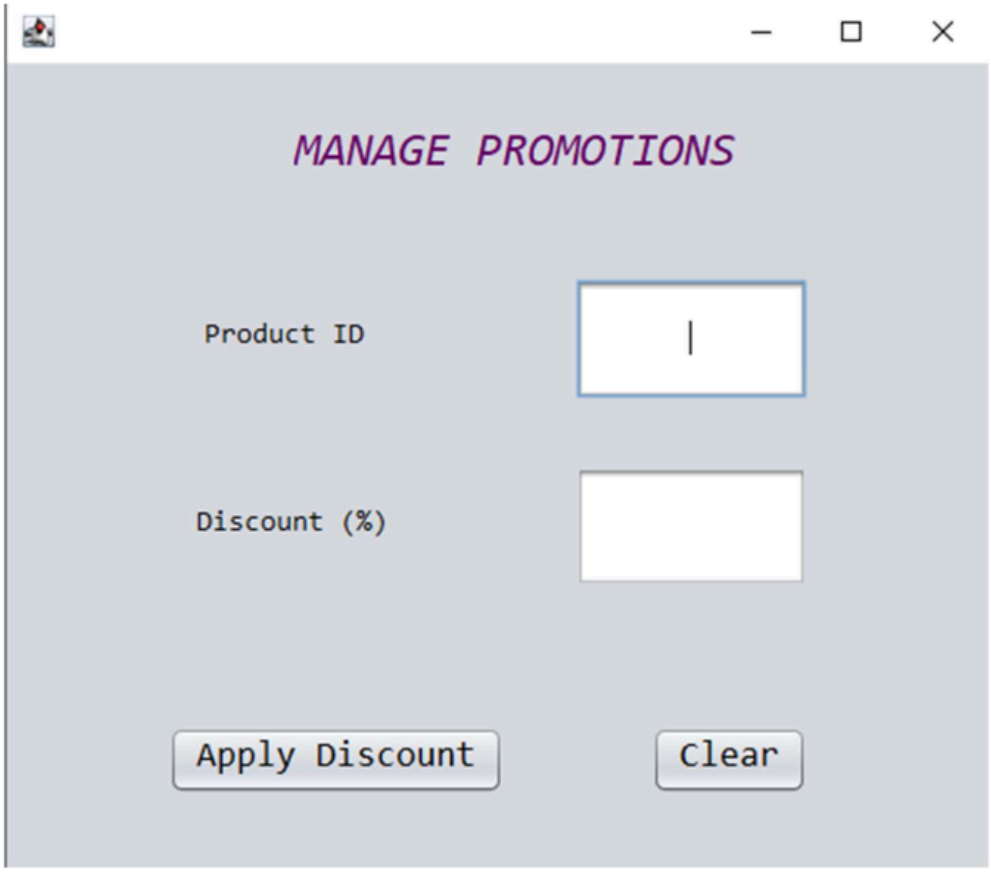


Figure 28 : Promotional Discount Interface

2B.9 Sales Summary

The Sales Summary function provides daily and monthly sales information.

SaleDAO performs aggregate database queries to calculate sales amounts and sales counts. The results are displayed through SalesSummaryForm.

This gives the Boutique Manager a simple overview of sales activity.

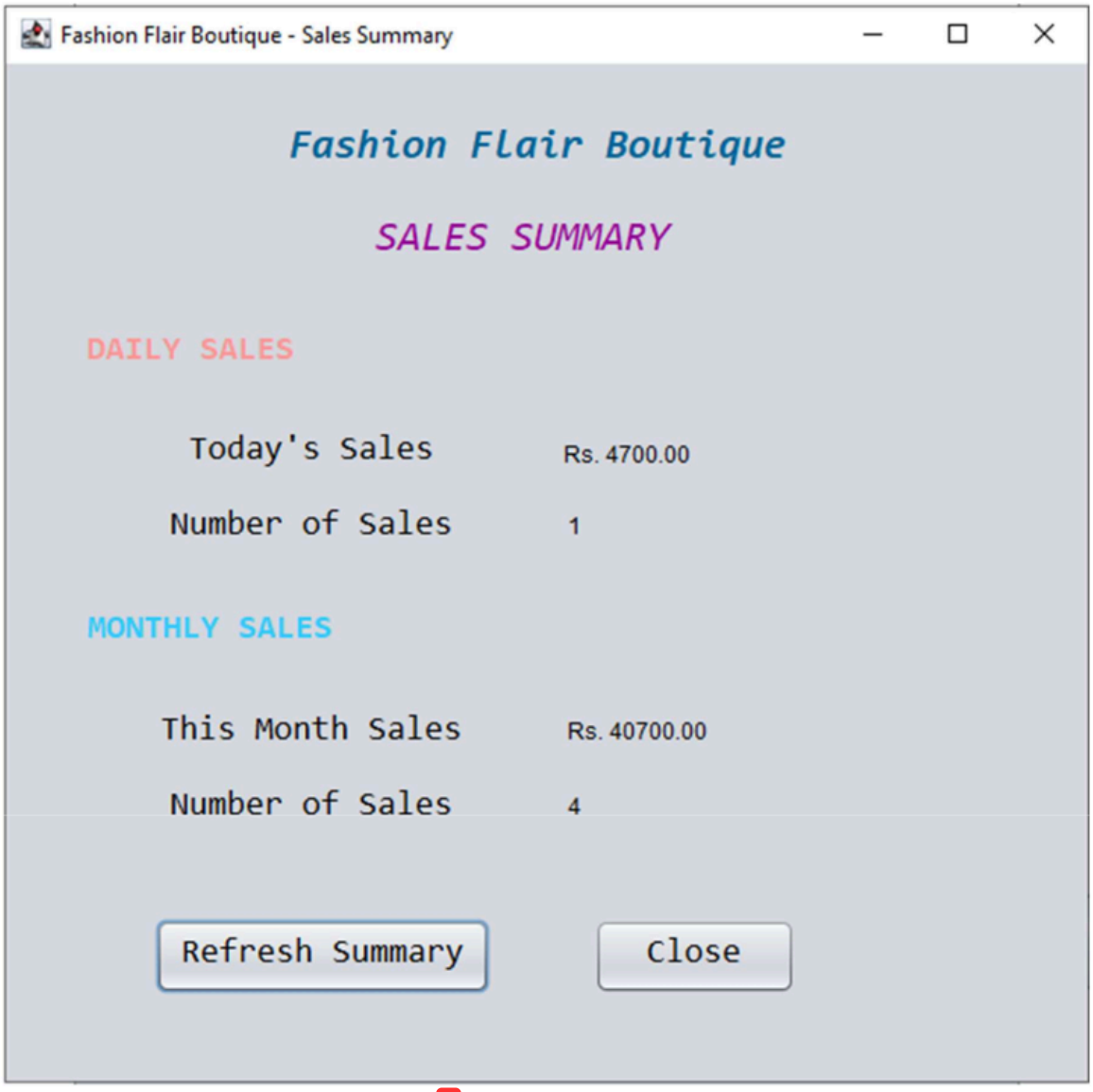


Figure 29 : Daily and Monthly Sales Summary

2C. Object-Oriented Concepts Applied in the Developed Program

The project demonstrates OOP through the combination of classes, objects, encapsulation, inheritance, polymorphism and abstraction.

The User class provides common user attributes and behaviour. SalesAssistant and BoutiqueManager inherit from it. This reduces duplication because common user information does not need to be written separately in both subclasses.

The Product class represents individual products. Product objects are created when data is entered or retrieved from the database.

Encapsulation is demonstrated through private fields and public getter/setter methods. This protects object data and provides controlled access.

Polymorphism is demonstrated by overriding login() in both user subclasses. A User reference can represent either a Sales Assistant or Boutique Manager object.

Abstraction is demonstrated using the Manageable interface. The interface defines common product operations while the Product class provides their implementation.

The DAO classes also contribute to the object-oriented structure because database responsibilities are separated from model classes and forms. For example, ProductDAO handles product database operations while Product represents product data.

2D. Database Operations

2D.1 Database Connection

The project uses Microsoft Access as the database and UCanAccess as the JDBC driver. The database file is named FashionFlairBoutique.accdb.

The DBConnection class is responsible for creating the database connection. It loads the UCanAccess driver, checks whether the database file exists and creates a JDBC connection.

DBConnection.java

Source | History

```java
package database;

import java.sql.Connection;
import java.sql.DriverManager;
import java.io.File;

public class DBConnection {

    private static Connection connection;

    public static Connection getConnection() {

        try {

            // Load UCanAccess JDBC Driver
            Class.forName("net.ucanaccess.jdbc.UcanaccessDriver");

            // Database file
            File databaseFile = new File("FashionFlairBoutique.accdb");

            // Check whether database exists
            if (!databaseFile.exists()) {

                System.out.println("Database file not found!");
                System.out.println(
                    "Expected location: "
                    + databaseFile.getAbsolutePath()
```

DBConnection.java

Source | History

```java
                );

                return null;
            }

            String databasePath = databaseFile.getAbsolutePath();

            String url = "jdbc:ucanaccess://" + databasePath;

            connection = DriverManager.getConnection(url);

            System.out.println("Database Connected Successfully!");

            return connection;

        } catch (ClassNotFoundException e) {

            System.out.println("UCanAccess Driver Not Found!");
            e.printStackTrace();

            return null;

        } catch (Exception e) {

            System.out.println("Database Connection Failed!");
            e.printStackTrace();

```

```
55                  return null;
56              }
57          }
58      }
```

Figure 30 : Database Connection Class

2D.2 Database Connection Test

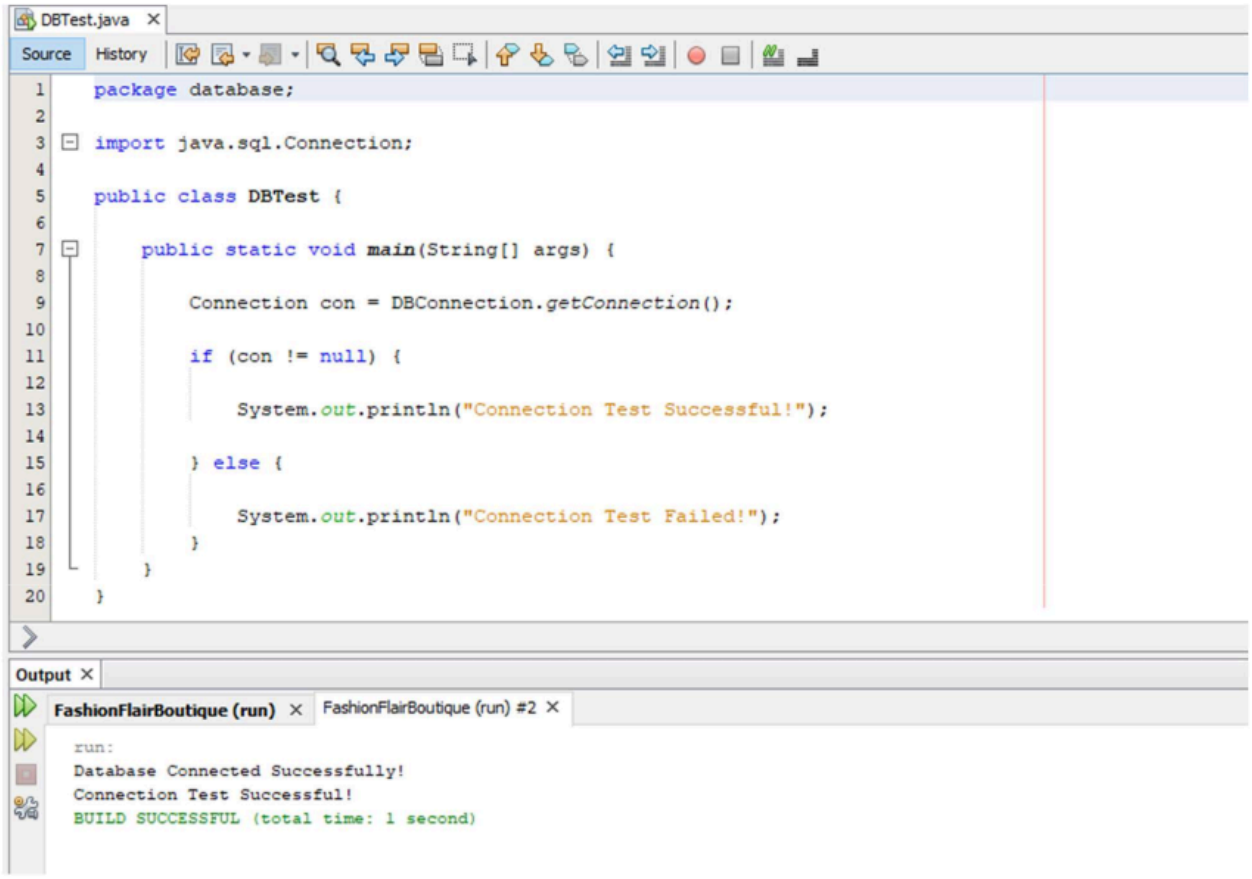


Figure 31 : Database Connection Test

2D.3 Database Connection Class

The system performs INSERT, UPDATE and DELETE operations.

When a new product is added, an INSERT query is executed through ProductDAO. When product information is modified, an UPDATE query is executed. The DELETE operation removes a selected product using its Product ID.

User account creation also uses INSERT, while account management can use UPDATE and DELETE operations.

Prepared statements are used for these operations. Parameterized queries reduce the possibility of SQL injection and make it easier to send user-entered values to the database.

2D.4 Data Retrieval

SELECT queries are used to retrieve products, users, categories and sales information.

For example, ProductDAO.getAllProducts() retrieves product records and converts each database row into a Product object. These objects are stored in an ArrayList.

Category searching uses a SELECT query with a category ID. User login also retrieves user details based on username and password.

2D.5 Database Management

The database contains the following major tables:

Table	Purpose
Users	Stores user account and role information
Categories	Stores fashion categories

Products	Stores product information and stock
Sales	Stores completed sales
SaleItems	Stores products included in each sale

Table 1 : Database Roles

Primary keys identify individual records. Foreign-key fields connect related records, such as Products.CategoryID, Sales.UserID, SaleItems.SaleID and SaleItems.ProductID.

The application also uses transactions when processing purchases. This is important because a purchase affects multiple database records.

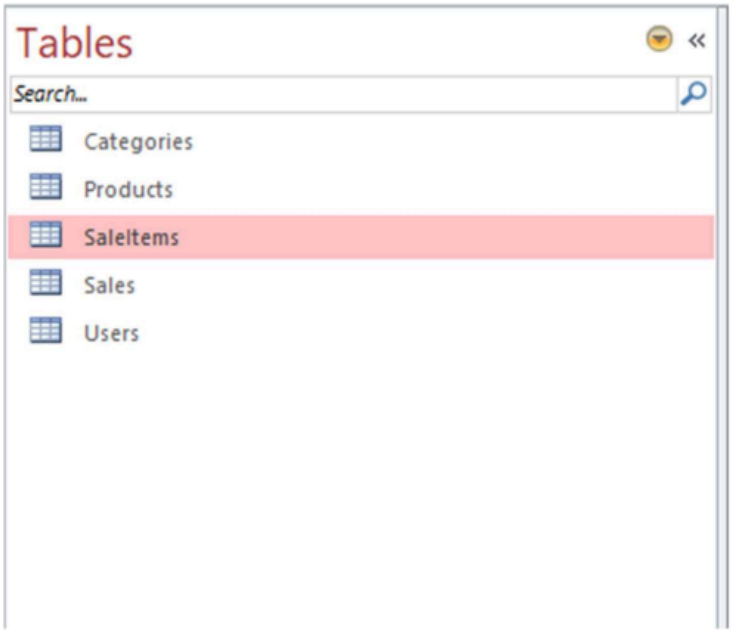


Figure 32 : Fashion Flair Boutique Database Tables

2E. User Manual

2E.1 Starting the System

The user first runs the Java application through NetBeans. The Login Form is displayed.

The user enters the username and password and selects Login. If the credentials are correct, the system identifies the user's role and opens the appropriate dashboard.

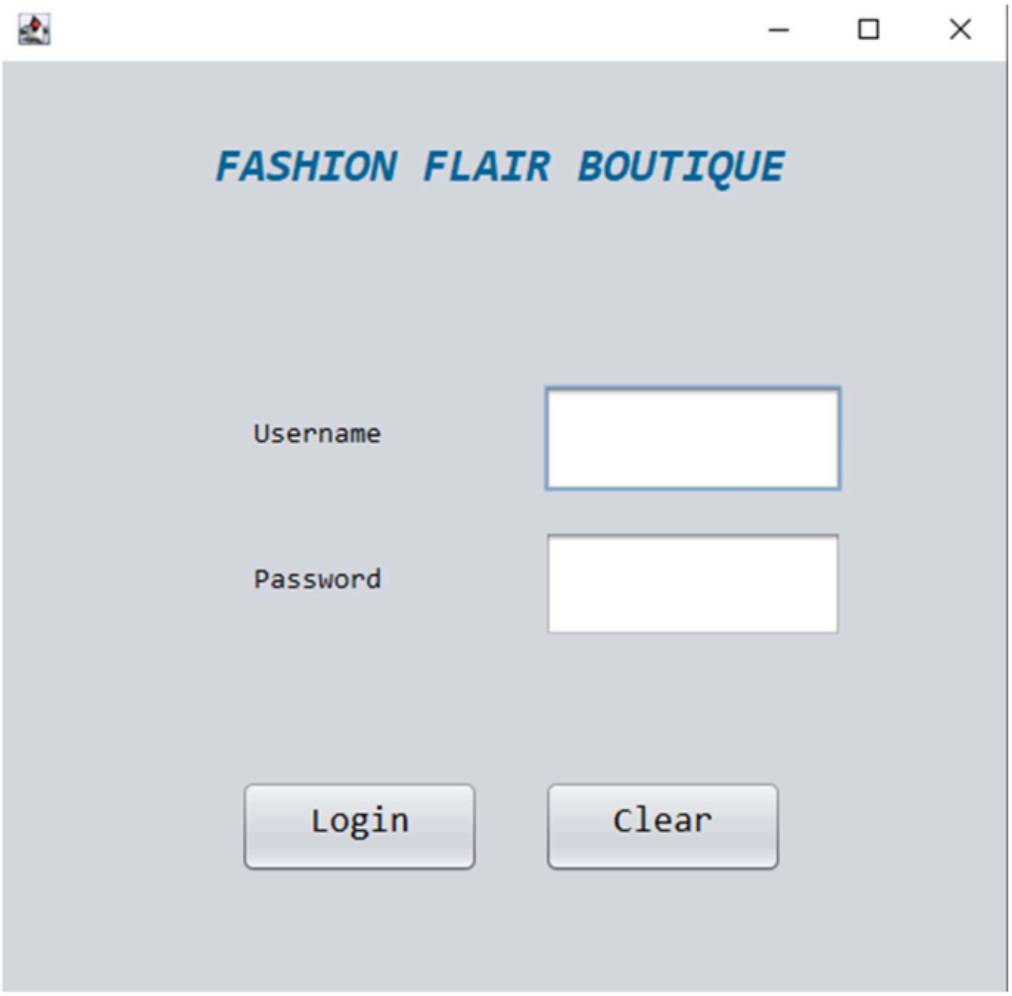


Figure 33 : Login Form

2E.2 Sales Assistant Instructions

After logging in as a Sales Assistant, the user can:

- Select View Products to view product details.
- Select Add Product to add a new fashion item.
- Select Search by Category to find products according to category.
- Select Process Purchase to record a customer purchase.
- Select Logout to return to the login screen.

The Sales Assistant should enter valid information into the forms and follow the confirmation messages displayed by the application.

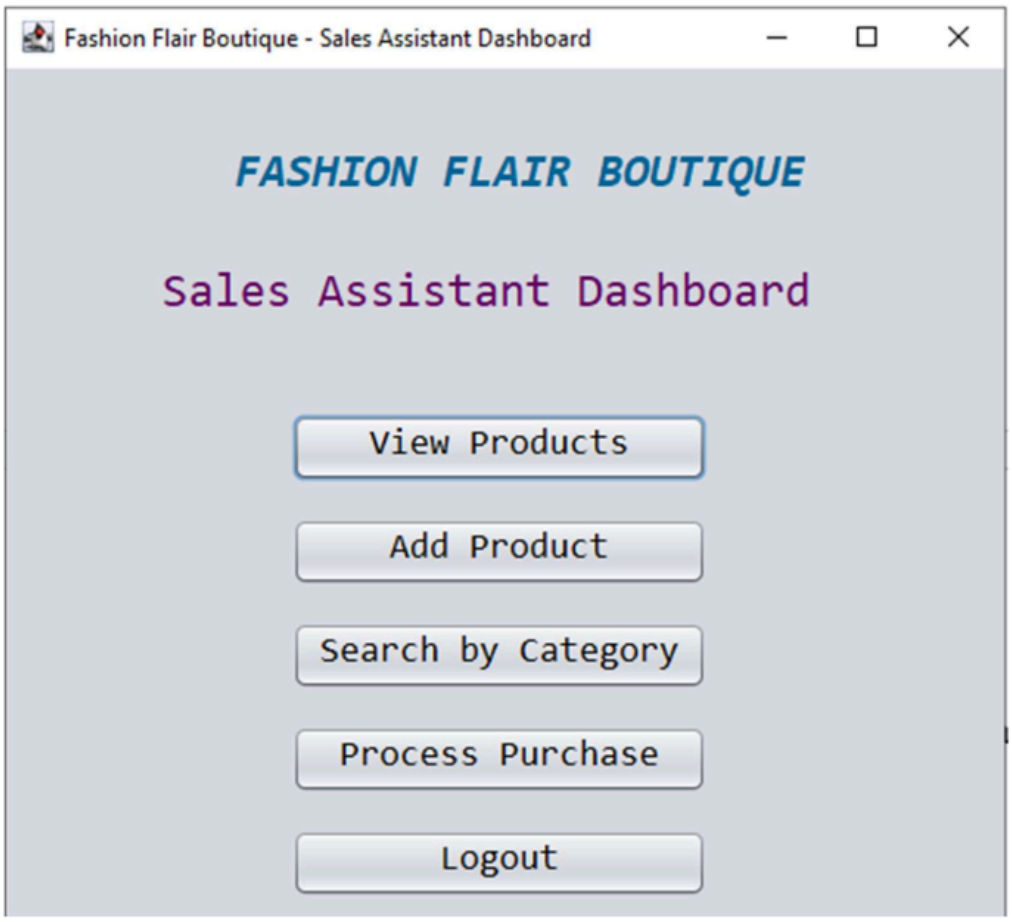


Figure 34 : Sales Assistant Dahboard

2E.3 Boutique Manager Instructions

After logging in as Boutique Manager, the user can access all Sales Assistant functions.

The Manager Dashboard provides:

- Manage Products
- Process Purchase
- Search by Category
- Manage Staff
- Update Prices
- Promotions
- Sales Summary
- Logout

Manage Products is used for viewing, adding, updating and deleting products.

Manage Staff is used to create and manage Sales Assistant accounts.

Update Prices is used to change the price of a selected product.

Promotions is used to apply percentage discounts.

Sales Summary is used to view daily and monthly sales information.



Figure 35 : Boutique Manager Dashboard

TASK 3 - Data Structures Used

Data structures are used to organize and manage data so that a program can process information effectively. In the Fashion Flair Boutique Management System, data structures are important because the system handles different types of information such as products, users, categories and sales records.

The developed Java application uses ArrayList, objects, ResultSet, JTable and DefaultTableModel for different purposes. The database is used for permanent storage, while Java data structures are used to handle information during program execution.

The selection of these structures was based on the requirements of the system, ease of implementation, efficiency and the ability to handle changing amounts of data.

3.1 Array List

The main collection data structure used in the system is Array List. It is used to store multiple objects dynamically.

In the developed system, ProductDAO retrieves products from the database and stores them in an ArrayList<Product>. UserDAO also uses an ArrayList to store Sales Assistant records retrieved from the database.

This is useful because the number of products and users is not fixed. For example, the boutique can add new products in the future. Therefore, a dynamic collection is more suitable than a fixed-size array.

The retrieved products can also be processed using a loop before displaying them in the user interface.

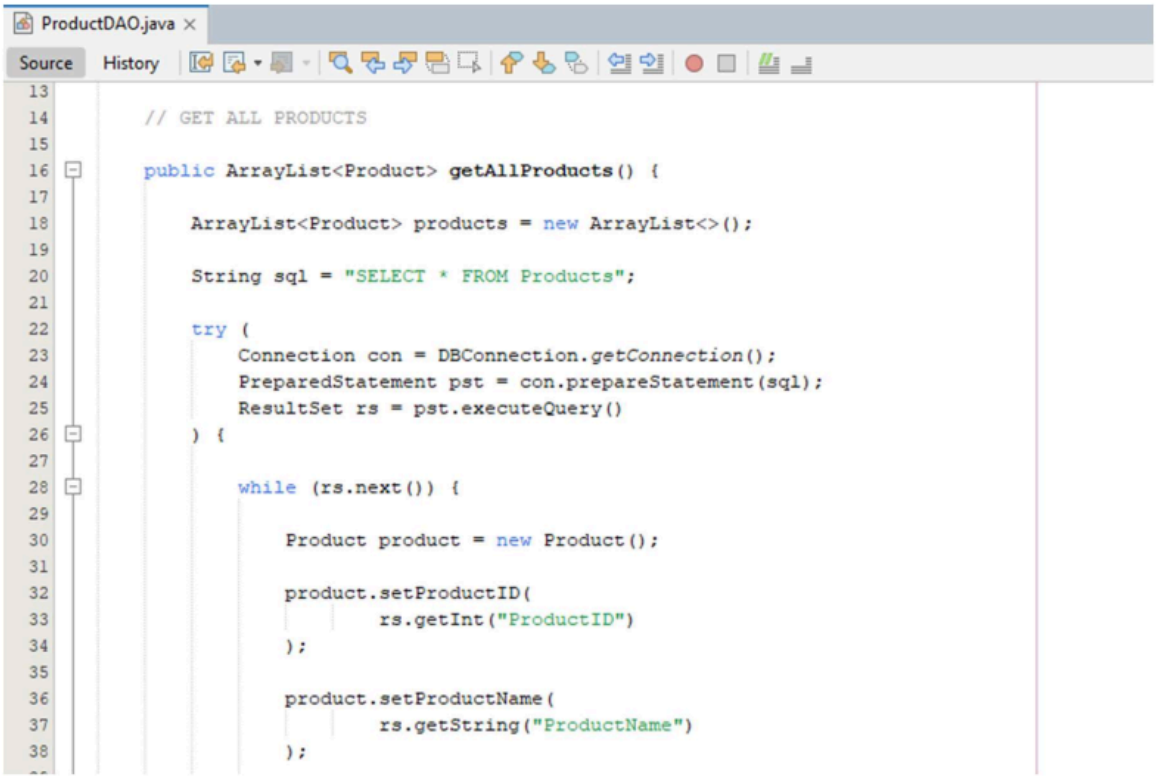


Figure 36 : ArrayList Used for Product Records

Suitability and Efficiency

ArrayList provides direct access to elements using an index and supports dynamic growth. It is also simple to use with Java objects. For the current boutique system, the number of products is expected to be manageable, making ArrayList an appropriate choice.

3.2 Objects

Objects are used throughout the system to represent real-world entities. The main examples are Product and User objects.

A Product object represents one fashion product and contains information such as Product ID, product name, brand, category, size, colour, price and stock quantity. A User object represents a person using the system.

When product information is retrieved from the database, the values are assigned to a Product object. These objects can then be stored inside an ArrayList.

Using objects keeps related information together and supports the Object-Oriented Programming design of the system.

Product.java

Source History

```
1  package model;
2
3  public class Product implements Manageable {
4      @Override
5  public void add() {
6      System.out.println("Product added successfully.");
7  }
8
9  @Override
10 public void update() {
11     System.out.println("Product updated successfully.");
12 }
13
14 @Override
15 public void delete() {
16     System.out.println("Product deleted successfully.");
17 }
18
19     private int productID;
20     private String productName;
21     private String brand;
22     private int categoryID;
23     private String size;
24     private String color;
25     private double price;
26     private int stockQuantity;
27
28     public Product() {
29     }
30
31     public Product(int productID, String productName, String brand,
32                    int categoryID, String size, String color,
33                    double price, int stockQuantity) {
34
35         this.productID = productID;
36         this.productName = productName;
37         this.brand = brand;
38         this.categoryID = categoryID;
39         this.size = size;
40         this.color = color;
41         this.price = price;
```

Figure 37 : Product Object Structure

Suitability

Objects are suitable because the system is based on OOP. Instead of handling each product attribute separately, all information belonging to one product can be represented through a single Product object. This improves organization, readability and maintainability.

3.3 Result Set

Result Set is used when information is retrieved from the Microsoft Access database through JDBC.

When a SELECT query is executed, the returned records are accessed using ResultSet. The program moves through the records using next() and retrieves individual values using methods such as getInt() and getString().

In the Fashion Flair Boutique system, ResultSet is used by DAO classes when retrieving products, users and other database information. The retrieved values are then used to create Java objects.

Therefore, ResultSet acts as a temporary structure between the database and the Java application.

ProductDAO.java

Source History

```java
package dao;

import database.DBConnection;
import model.Product;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.util.ArrayList;

public class ProductDAO {


    // GET ALL PRODUCTS

    public ArrayList<Product> getAllProducts() {

        ArrayList<Product> products = new ArrayList<>();

        String sql = "SELECT * FROM Products";

        try (
            Connection con = DBConnection.getConnection();
            PreparedStatement pst = con.prepareStatement(sql);
            ResultSet rs = pst.executeQuery()
        ) {

            while (rs.next()) {

                Product product = new Product();

                product.setProductID(
                        rs.getInt("ProductID")
                );

                product.setProductName(
                        rs.getString("ProductName")
                );

```

Figure 38 : ResultSet Used for Database Retrieval

Suitability and Efficiency

ResultSet is suitable because it is directly supported by JDBC. It allows database records to be processed sequentially without requiring a separate file-reading mechanism. It also works effectively with the DAO structure used in the project.

3.4 JTable and DefaultTableModel

The system uses JTable with DefaultTableModel to display product information in the graphical user interface.

The Manage Products form displays Product ID, Name, Brand, Category, Size, Colour, Price and Stock in rows and columns. Each row represents a product while each column represents a particular property.

DefaultTableModel manages the data contained in the table. When products are retrieved from the database, the program adds the product information as rows to the table.

This structure is especially suitable for an inventory management system because users need to view several product records at the same time.

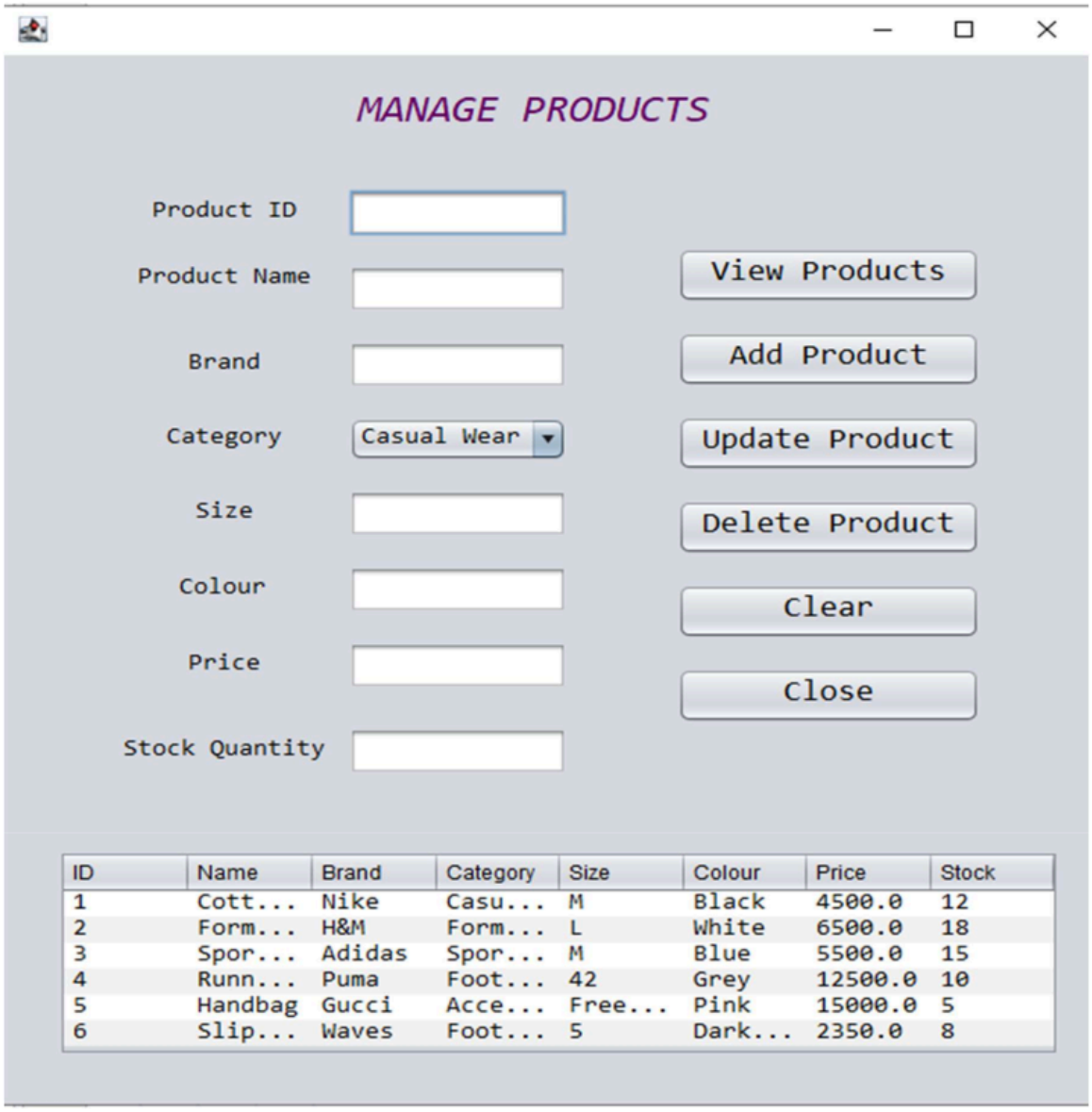


Figure 39 : Product Records Displayed Using JTable

Suitability

JTable was selected because it provides an organized and user-friendly method of displaying multiple records. It also allows the user to select a particular product row, which is useful for updating or deleting products.

3.5 Comparison of Data Structures

Different data structures could have been used to handle the information in the application. However, the selected structures are suitable for the specific requirements of this project.

A fixed-size array would require the programmer to define the number of elements in advance. This is not ideal because the boutique can add or remove products. ArrayList avoids this limitation by allowing the collection to grow dynamically.

A LinkedList could also store product objects, but the current application mainly retrieves and displays product records. It does not require frequent insertion or deletion in the middle of the collection. Therefore, ArrayList provides a simpler solution.

A HashMap would be useful if the system frequently needed to retrieve objects directly using a unique key. However, the current product functions mainly retrieve collections of products and display them, so ArrayList is suitable for the implemented requirements.

The database is used instead of Java collections for permanent storage because product, user and sales information must remain available after the application closes.

3.6 Efficiency, Scalability and Suitability

The selected data structures support the main requirements of the system.

Efficiency:

ArrayList provides efficient access to stored elements, while ResultSet provides a direct way to process database query results. JTable allows retrieved information to be displayed without requiring a separate display mechanism.

Scalability:

The number of products and users can increase over time. ArrayList can dynamically accommodate additional objects, while the Access database provides persistent storage for growing records.

Suitability:

Each structure has a specific purpose. Objects represent business entities, ArrayList stores collections of objects, ResultSet handles database query results, and JTable displays records to users. Therefore, the structures work together rather than performing the same task.

Conclusion

The Fashion Flair Boutique system was developed as an automated solution for managing products, users, purchases and sales information. The system addresses the main requirements of the given scenario by providing separate functions for Sales Assistants and Boutique Managers.

The project demonstrates practical application of Object-Oriented Programming concepts. Classes represent system entities, objects represent individual records, encapsulation protects object data, inheritance creates relationships between user roles, polymorphism allows different implementations of common behaviour and abstraction defines common operations through an interface.

The application also separates responsibilities using model, DAO and user interface classes. This structure makes the system easier to understand and maintain. Database operations are handled using UCanAccess and Microsoft Access, allowing information to be stored and retrieved between application sessions.

The developed system includes product management, category searching, customer purchase processing, staff account management, price updating, promotional discounts and sales summaries. These functions provide the main operational features required by the boutique scenario.

The use of ArrayList provides dynamic handling of product and user collections, while JTable and DefaultTableModel provide an effective way of presenting retrieved records to users.

Overall, the project demonstrates how OOP concepts can be applied to a realistic business problem while integrating a database for persistent data management. The implementation provides a suitable foundation for a small boutique management system and demonstrates the practical relationship between object-oriented design, database operations and user interface development.

References

- Deitel, P. and Deitel, H. (2020) Java: How to Program, Early Objects. 11th edn. Pearson.
- Eckel, B. (2006) Thinking in Java. 4th edn. Prentice Hall.
- GeeksforGeeks (2024) Object Oriented Programming (OOPs) Concept in Java. Available at: https://www.geeksforgeeks.org/java/object-oriented-programming-oops-concept-in-java/ (Accessed: 16 September 2026).
- GeeksforGeeks (2024) ArrayList in Java. Available at: https://www.geeksforgeeks.org/java/arraylist-in-java/ (Accessed: 16 September 2026).
- Microsoft (2024) Microsoft Access Documentation. Available at: https://support.microsoft.com/access (Accessed: 16 September 2026).
- Oracle (2024) Java Platform, Standard Edition Documentation. Available at: https://docs.oracle.com/en/java/javase/ (Accessed: 16 September 2026).
- Oracle (2024) The Java Tutorials: Object-Oriented Programming Concepts. Available at: https://docs.oracle.com/javase/tutorial/java/concepts/ (Accessed: 16 September 2026).
- Oracle (2024) The Java Tutorials: Interfaces. Available at: https://docs.oracle.com/javase/tutorial/java/IandI/createinterface.html (Accessed: 16 September 2026).
- Oracle (2024) The Java Tutorials: Inheritance. Available at: https://docs.oracle.com/javase/tutorial/java/IandI/subclasses.html (Accessed: 16 September 2026).
- Oracle (2024) The Java Tutorials: JDBC Basics. Available at: https://docs.oracle.com/javase/tutorial/jdbc/basics/ (Accessed: 16 September 2026).
- Oracle (2024) The Java Tutorials: Creating a GUI With Swing. Available at: https://docs.oracle.com/javase/tutorial/uiswing/ (Accessed: 16 September 2026).
- Schildt, H. (2018) Java: The Complete Reference. 11th edn. McGraw-Hill Education.

- UCanAccess (2024) UCanAccess: JDBC Driver for Microsoft Access. Available at: https://ucanaccess.sourceforge.net/site.html (Accessed: 16 September 2026).
- W3Schools (2024) Java Classes and Objects. Available at: https://www.w3schools.com/java/java_classes.asp (Accessed: 16 September 2026).
- W3Schools (2024) Java Data Structures. Available at: https://www.w3schools.com/java/java_data_structures.asp (Accessed: 16 September 2026).

Buddhini Ishani OOP.docx

ORIGINALITY REPORT

18%	7%	3%	17%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to University of Wales Institute, Cardiff Student Paper	13%
2	Submitted to Erasmus University Rotterdam Student Paper	2%
3	Submitted to UNICAF Student Paper	1%
4	Submitted to Universiti Teknologi MARA Student Paper	1%
5	Submitted to AlHussein Technical University Student Paper	<1%
6	Submitted to University of Lincoln Student Paper	<1%
7	Submitted to World Citi Colleges-Quezon City Student Paper	<1%
8	Submitted to Asia Pacific University College of Technology and Innovation (UCTI) Student Paper	<1%
9	Submitted to Zambia Centre for Accountancy Studies Student Paper	<1%
10	www.coursehero.com Internet Source	<1%
11	datadrivensurveillance.org Internet Source	<1%
12	stannscollegehyd.com Internet Source	<1%

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	Off		